

AI Application Specialist

대규모 언어모델 이해 및 파인튜닝

2026

사내교수 박영진 / 첨단기술아카데미

“ LLM 의 기본 구조를 이해하고

이해한 것은 코드로 실습해서

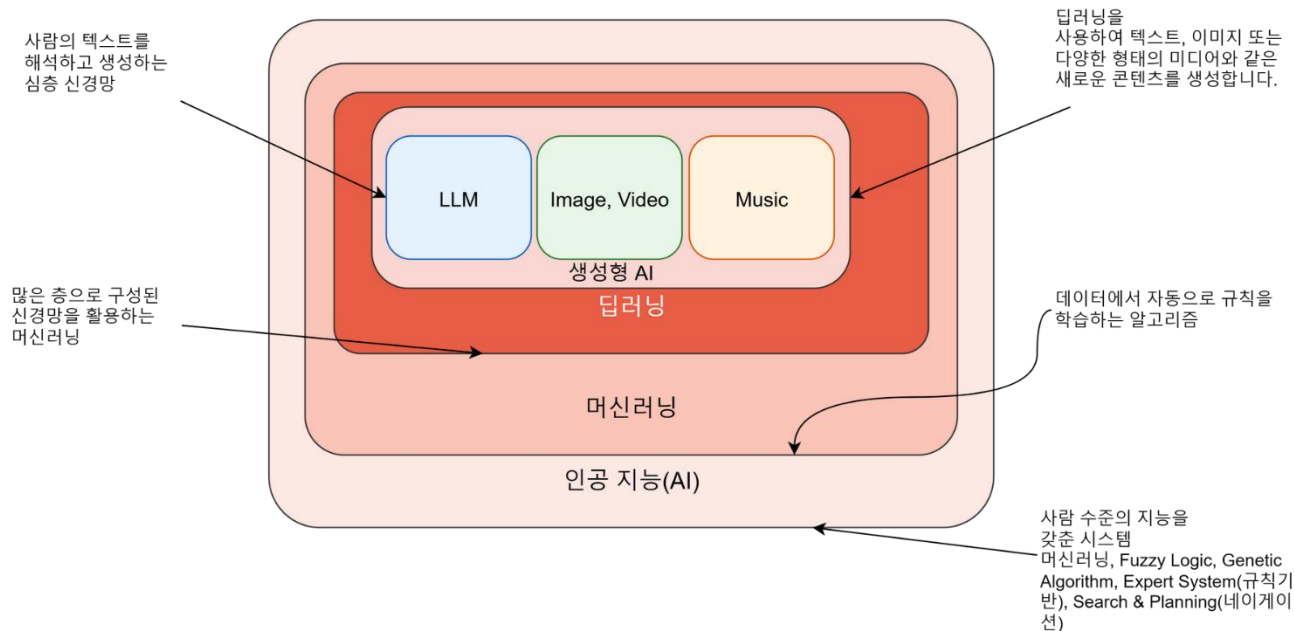
LLM을 잘 사용하는 것”

Schedule

시간	1 일차	2 일차
08:30 ~ 09:20	• 과정 OT: 과정 소개 / AVD 소개	• GPT 모델
09:30 ~ 10:20	• AI 모델의 역사와 미래	• GPT 모델 실습
10:35 ~ 11:15	• 수학적 사전지식	• Base 모델 학습
11:30 ~ 12:00	• 언어 모델의 발전과 실습	• Base 모델 실습
12:00 ~ 13:15	중식	
13:15 ~ 14:20	• 데이터입력과 모델 학습	• Category 분류 Fine Tuning
14:30 ~ 15:20	• Dataset 실습	• Category 분류 Fine Tuning 실습
15:30 ~ 16:20	• Attention	• Instruction Following Fine Tuning
16:30 ~ 17:20	• Attention 실습	• 최신 모델 최적화 기법

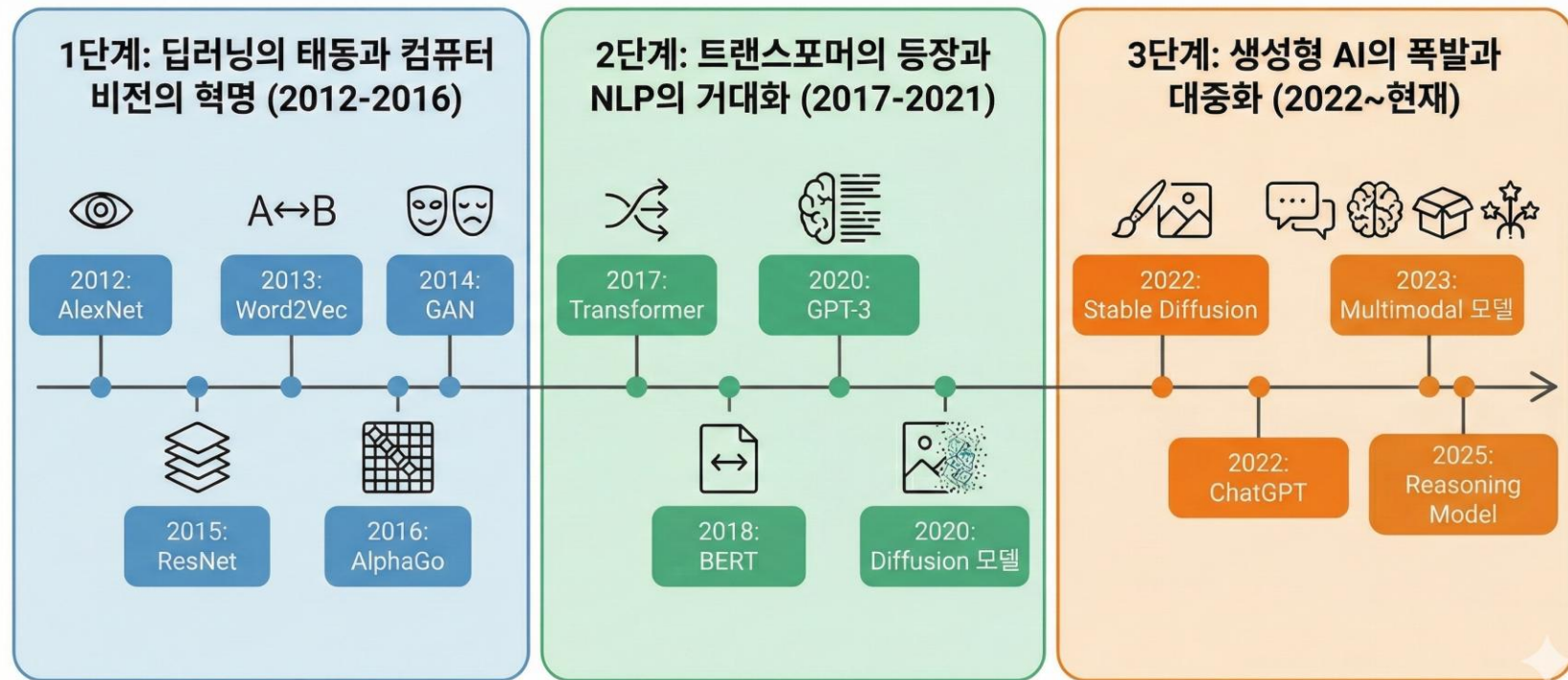
AI 모델의 역사

✓ 머신러닝 -> 딥러닝 -> 생성형 AI



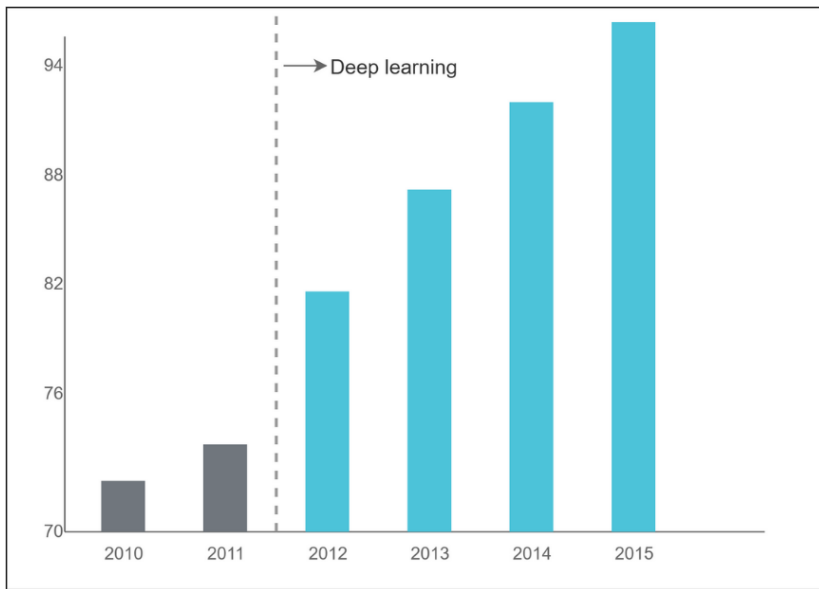
Deep Learning 모델의 역사

✓ Deep Learning 모델의 역사

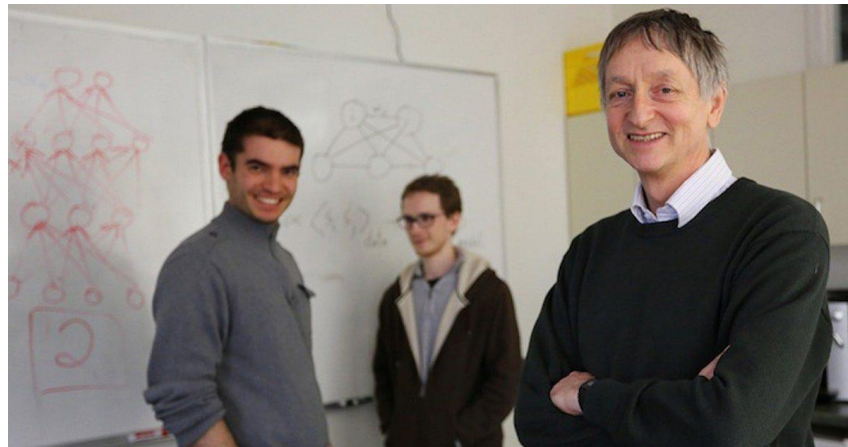


✓ ImageNet 대회에서 압도적으로 1등

- Deep Learning의 시작, GPU의 사용
- Layer을 깊게 쌓기 위한 경쟁 돌입



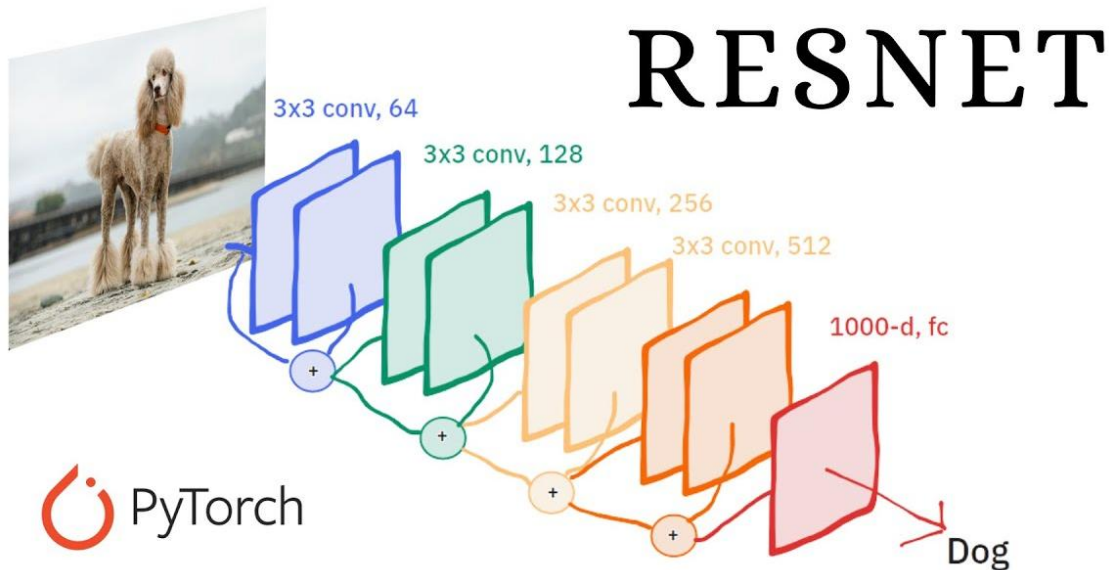
[AI 이야기] 인간 VS 인공지능 (3)딥러닝의 시대를 연 알렉스넷(AlexNet)



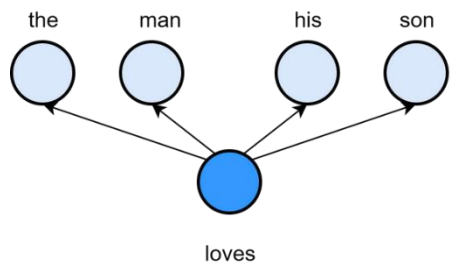
오른쪽부터 제프리 힌든, 알렉스 키르제브스키, 일리야 수츠케버

✓ 신경망의 깊이 한계를 깨부셨다

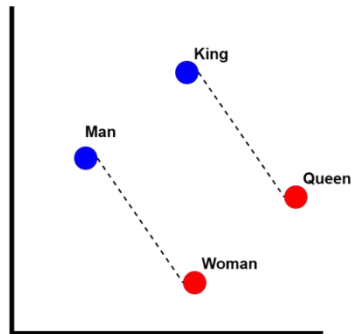
- Skip Connection 도입
- Layer을 깊게 쌓기 위한 한계 돌파



- ✓ 단어의 의미를 벡터 공간에 보존하여 단어 간의 연산을 가능하게 함(추론)



word2vec(2013)



토마시 미콜로프

단어의 의미를 벡터 공간에 맵핑하여 계산할 수 있게 됨

"This infuriated a local linguist who declared my ideas to be a total nonsense"

✓ AI의 미래를 보여주는 사례

AlphaGo



78수

알파고의 몬테카를로 트리 탐색
알고리즘이 예상치 못한 상황에서
어떻게 오류를 일으키는지 보여준
역사적인 사례

AlphaGo Zero



- 스스로 깨우침
- 독창적이고 효율적인 수법
- 간결한 하드웨어: TPU 4대로 3일간 학습

AlphaZero



- AlphaGo를 다른 게임으로 확대
- 짧은 학습 시간으로 압도함
- 규칙만으로 학습하는 범용성
- 공격적이고 창의적인 스타일

- ✓ "Attention is all you need"

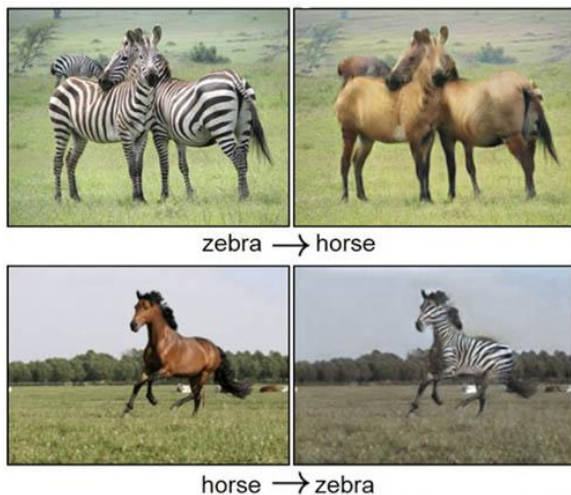
- 모든 저자들이 창업하여 수천에서 수조원의 스타트업 창업



✓ Multimodal 생성의 시대

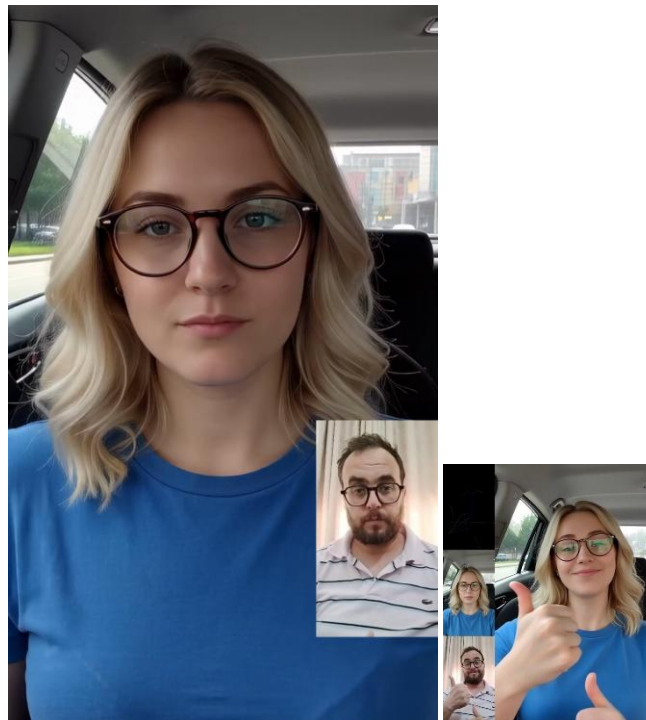


I, Robot(2004)



CycleGAN(2017)

[GAN이란? - 생성적 대립 신경망 설명 - AWS](#)

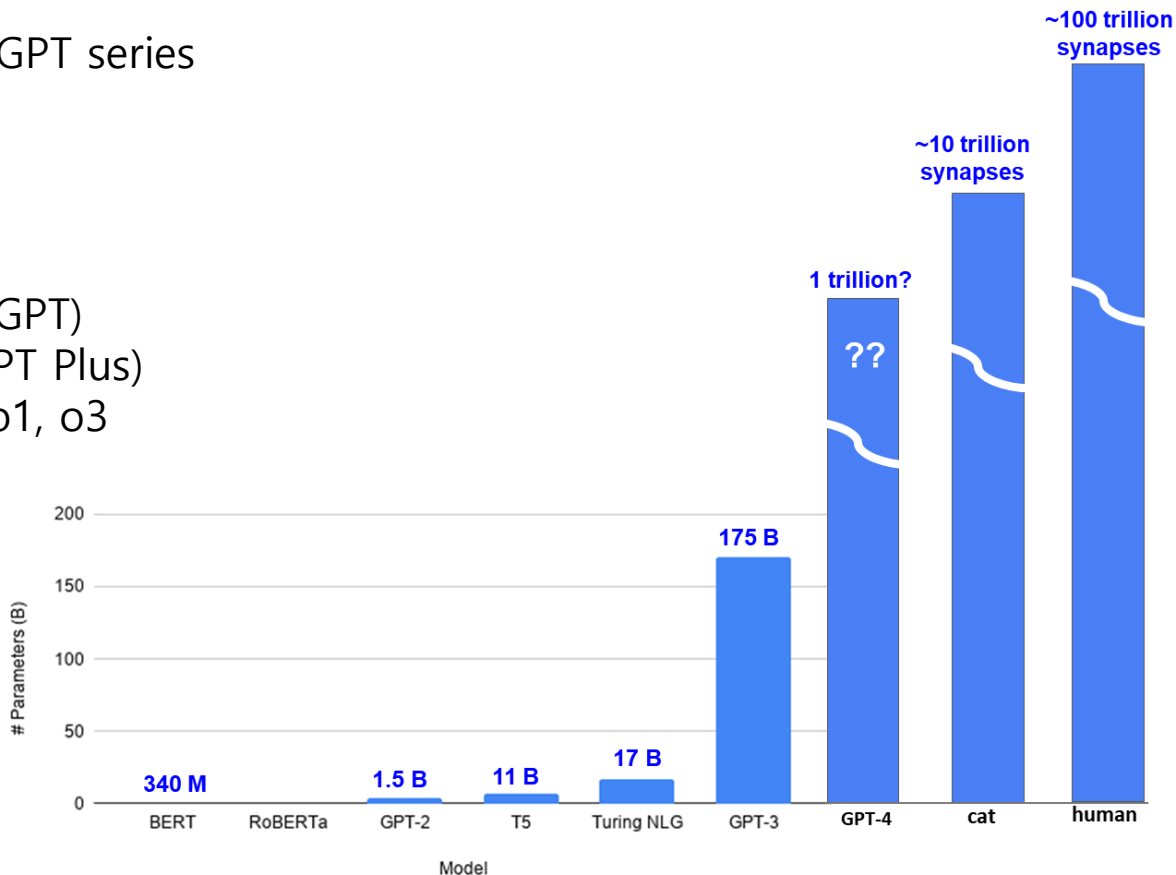


비디오 생성

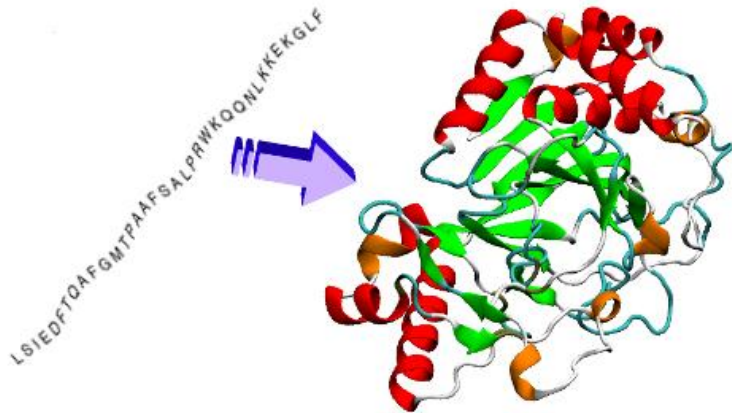
GPT: 규모의 시대

✓ Comparisons between GPT series

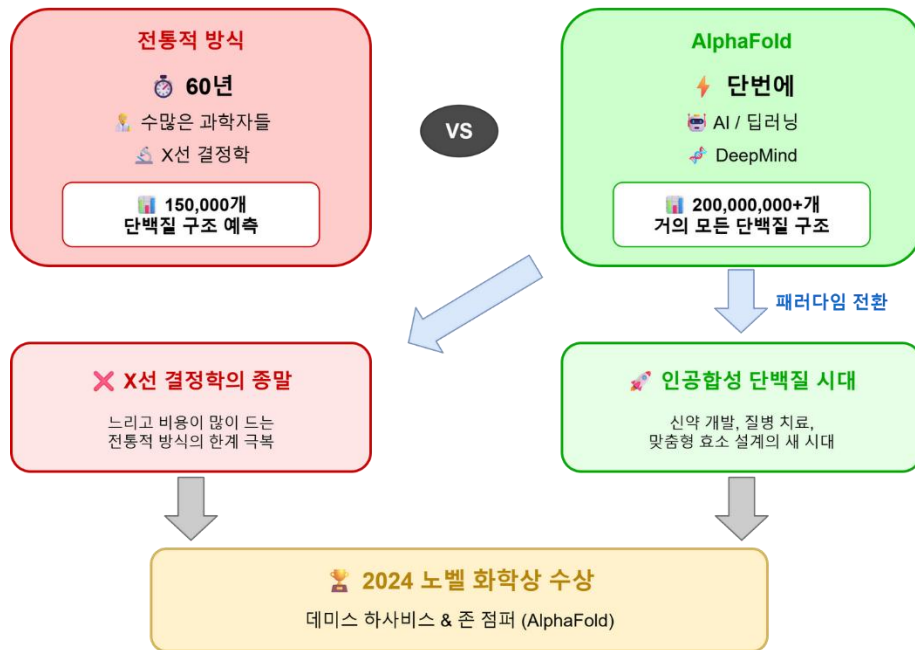
- 2018 GPT-1
- 2019 GPT-2
- 2020 GPT-3
- 2022 GPT-3.5 (ChatGPT)
- 2023 GPT-4 (ChatGPT Plus)
- 2024 GPT-4o, GPT o1, o3
- 2025 GPT-4.5
- 2025 GPT-5



✓ 단백질 구조



✓ 전통적 방식의 종말



AI의 미래

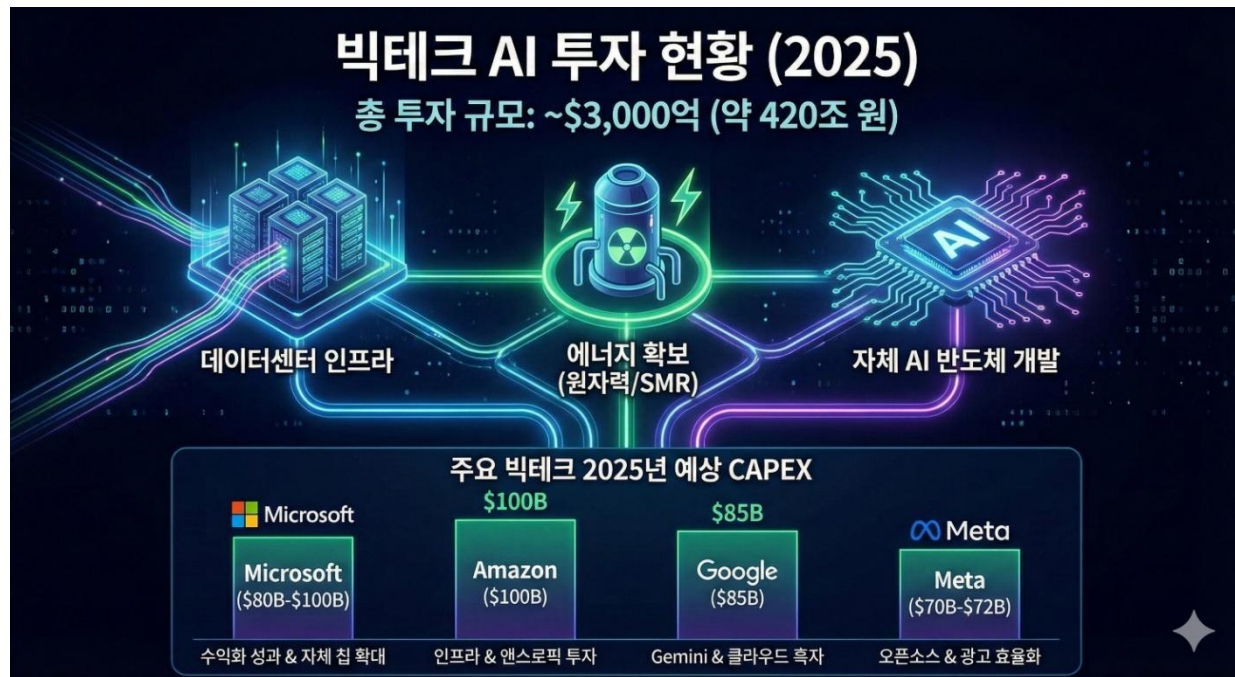
- ✓ 에너지, 인력, 돈, 자원을 빨아들이는 AI



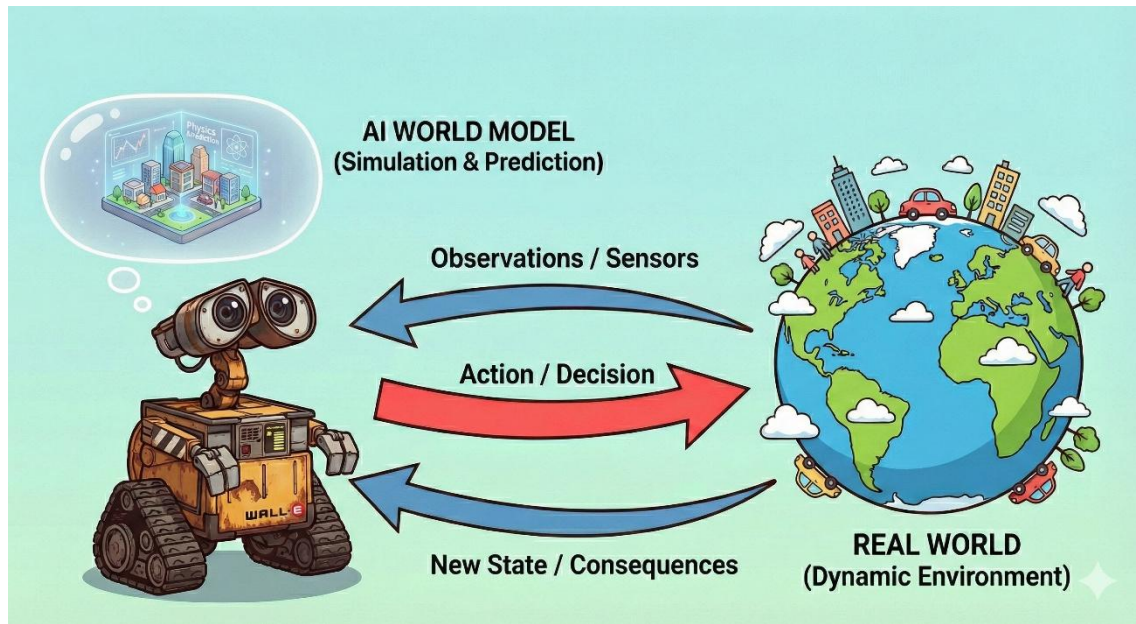
빅테크의 AI 투자 계획

✓ 4개 빅테크 회사들이 1년에 3000억 달러 규모 투자

- 누구도 멈출 수 없는 AI의 발전

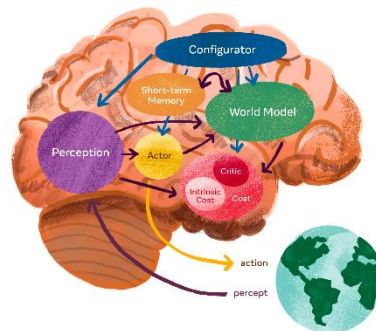
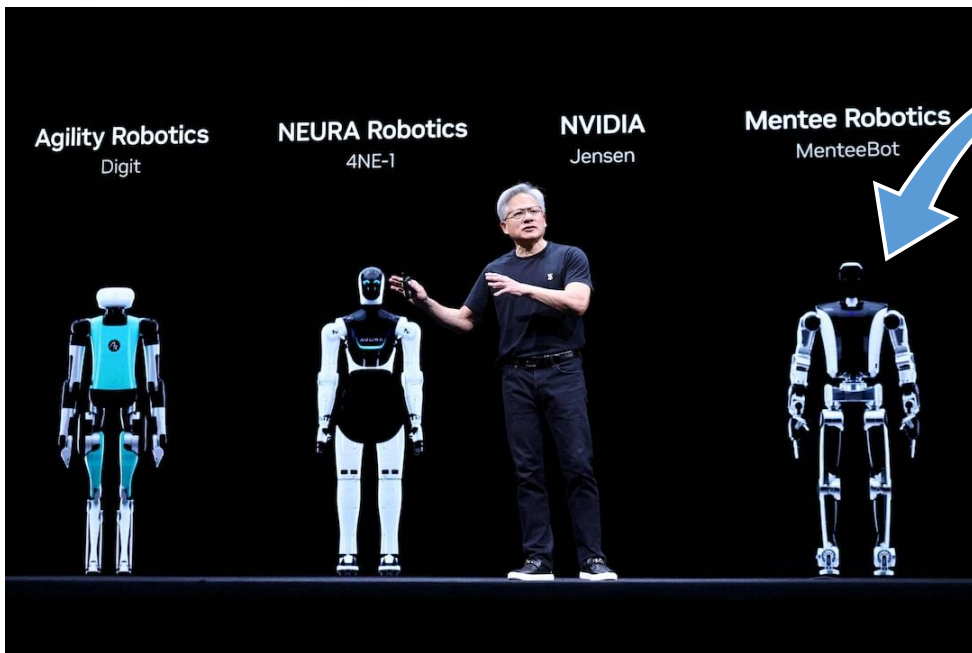


- ✓ 세계에 대한 내재적 모델을 가지고 물리적 세계의 원리를 이해하고 행동 결과를 예측하는 AI 시스템



Humanoid의 발전

- ✓ Humanoid에 world model을 넣으면? Embodied AI



[Exclusive: Nvidia, Foxconn in talks to deploy humanoid robots at Houston AI server making plant | Reuters](#)

✓ 인간과 침팬지

- 인간과 침팬지 > 침팬지와 오랑우탕



- 침팬지와 인간의 차이

	침팬지	인간	비고
집단크기	50개체	150개체	언어가 비접촉식 사회 연결을 가능하게 함
도구 개발	O	O	
지식 전파	X	O	인간은 지식을 가르치고 배울 수 있음
시뮬레이션	근미래와 경험	추상과 허구	

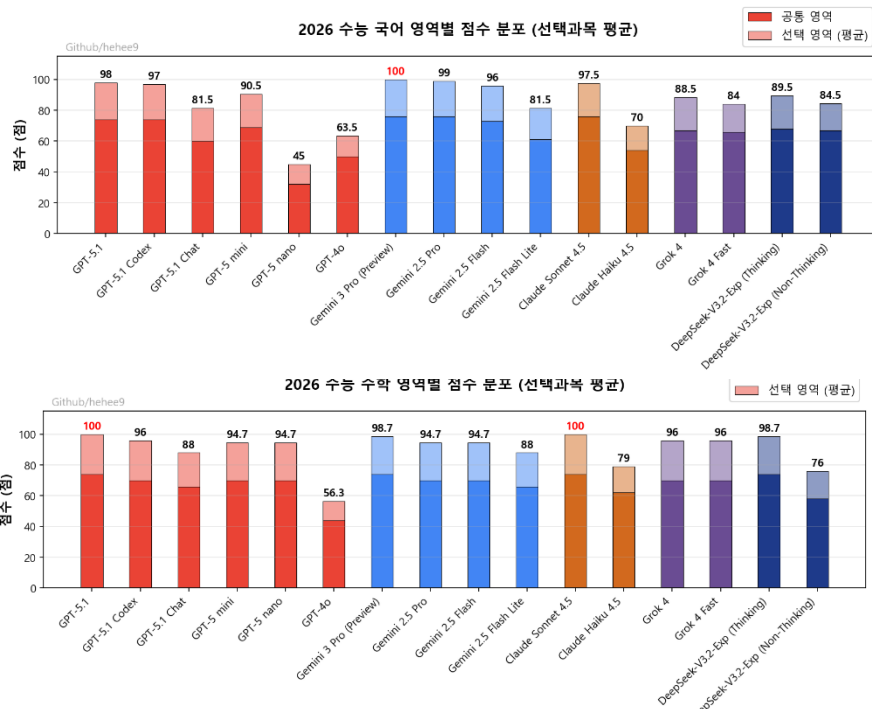
✓ 뇌의 가소성

- 경험과 환경에 따라 끊임없이 조정되고 재구성되는 능동적 시스템
- 태어났을 때 시각 장애인이었던 환자가 시각을 회복하면 볼 수 있을까?
- 어렸을 때 언어를 배우지 못한 소녀가 말을 할 수 있을까?
- 사회에 나가기까지 20년 이상의 배움이 필요하고 증가 추세

[인간 뇌의 가소성 - 고등과학원 HORIZON](#)

✓ 2026년도 수능에서 450점 만점에 440점

순위	모델명	원점수	추정 등급컷(2025.11.18기준)
🥇 1st	o1-Preview	97	1등급
🥈 2nd	o1-mini	78	4등급
🥉 3rd	gpt-4o	75	4등급
4th	gpt-4o-mini	59	5등급
5th	gpt-3.5-turbo	16	8등급



노동의 가치와 해방

✓ 인력 감축

- 노동 가치의 상실
- 올해 미국 빅테크 인력 감원 증가

기업	사례
마이크로소프트 Microsoft	·5월: 제품-엔지니어링 부서를 중심 6000명 해고 ·7월: 전체 직원 4%에 규모인 9000명 감원
메타 Meta	·2월: 저성과자 3600명 해고 ·10월: AI 부문 600명 감원
구글 Google	·2월: 클라우드 부문 100명 미만 감원 ·4월: 플랫폼-디바이스 부문 수백명 감원 ·5월: 판매-파트너십 부문 200명 해고 ·6월: 클라우드 부문 100명 이상 감원
아마존 amazon	·10월: 본사 인력 10% 규모인 3만명 감원 추진

자료: 각사 및 외신 보도 종합

The JoongAng

[AI '일자리 침공' 현실로, 아마존 3만명 줄인다 | 중앙일보](#)

✓ 노동의 해방

- 기본 소득 시대
- 인간은 노동으로부터 해방될 수 있을까?



[\[AI패권전쟁 한국의 승부수\]머스크도, 챗GPT 창시자도..."기본소득 시대 온다" - 아시아경제](#)

산업의 파괴

✓ Claude Cowork 출시

- 사용자의 PC에서 직접 업무를 수행
- SaaS, 법률 및 전문 서비스, 데이터분석, 사이버보안 업체의 주가 하락



<https://youtu.be/brrtbMmCkghM>

✓ Seedance 2.0 출시

- Bytedance에서 출시한 비디오 생성 모델
- 텍스트+이미지+오디오+영상을 입력받아 비디오를 생성



<https://youtu.be/0Bgzg3B-0bl>

✓ KAIST 킬러로봇 개발에 해외과학자 성명

- AI를 살상무기에 연구할 경우 연구 협업 보이콧



✓ Anthropic vs OpenAI

- 미국방부 AI 모델 100% 활용 요청
- Anthropic 거부, OpenAI 수용



의식과 자유의지

✓ 의식

- 구글의 한 개발자가 구글의 인공지능(AI) 챗봇 람다에 의식이 있다고 주장
- 통합 정보 이론: 어떤 시스템이 정보를 **통합**하는 능력이 있다면, 그 시스템은 그만큼의 **의식**을 가진다
- 인간의 뇌를 On/Off할 수 있는 Claustrium



[이슈] "람다, 너는 의식이 있어?" "응, 그런 것 같아"

✓ 자유의지

- 우리의 결정보다 뇌가 먼저 정했을까?(Libet, 1983)
- \$1,000를 어느 자선 단체에 기부할지 고민해서 선택(Uri Maoz et al., 2019)
- 가치 있고 신중한 선택에서는 전위가 나타나지 않음



우리의 결정보다 먼저 뇌에서 벌어지는 일 | 세상의 결말이 다 정해져 있는 걸까..?

새로운 지적인 존재

✓ 현생 인류가 외계인보다 먼저 맞이하는 지적인 존재

- 의식과 자유 의지를 가질까?



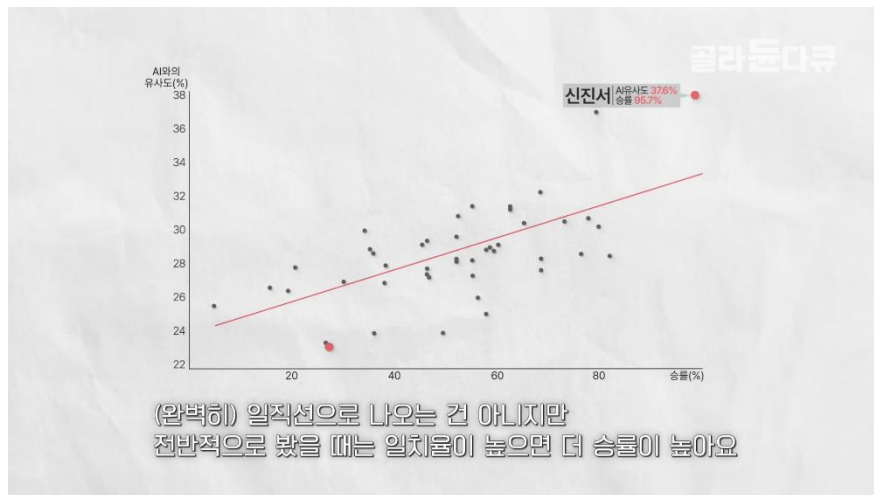
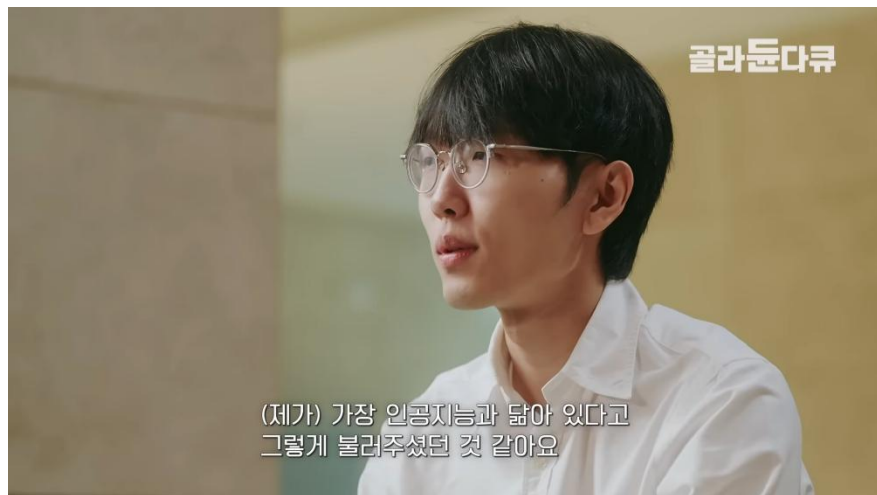
A long time ago in a galaxy far,
far away....

[Why are we only left among human races?](#)

[영 학자 "외계인 못 만나는 이유는 AI" : 네이버 블로그](#)

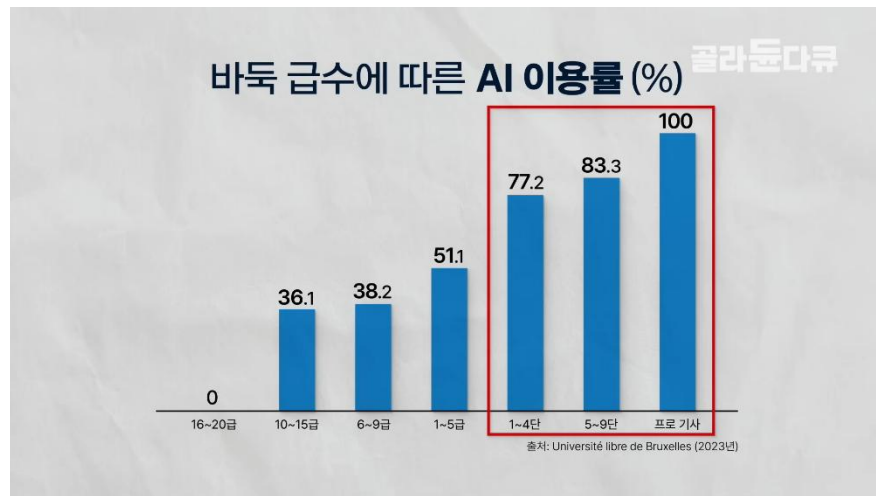
AlphaGo 이후

✓ AI에서 배우는 바둑



[이젠 옛날로 돌아갈 수 없다. 모든 것이 인공지능인 바둑계 충격적인 근황 | 알파고 10주년 | AI | 신진서 | 이세돌 | 다큐프라임 | #골라둔다큐](#)

- ✓ 전문가일수록 더 잘 활용



[이젠 옛날로 돌아갈 수 없다. 모든 것이 인공지능인 바둑계 충격적인 근황 | 알파고 10주년 | AI | 신진서 | 이세돌 | 다큐프라임 | #골라둔다큐](#)

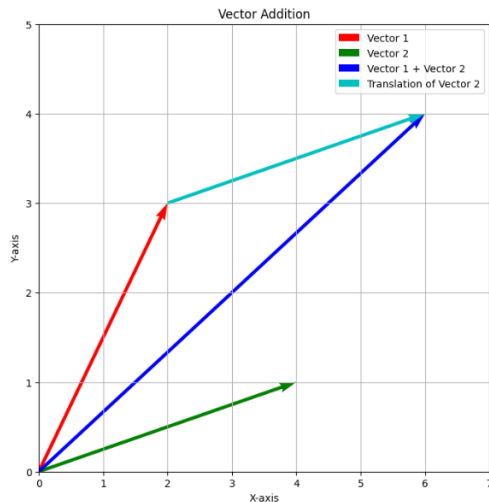
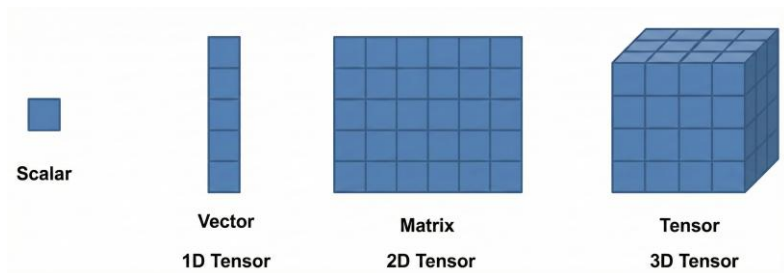
- ✓ AI를 어떻게 바라봐야 할까



수학적 사전지식

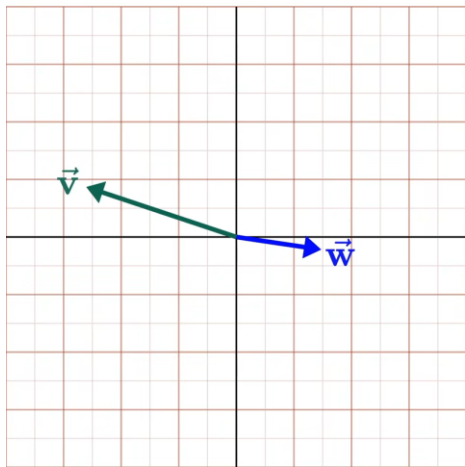
벡터: 덧셈

- ✓ 좌표 공간 상에서 한 점을 향하는 화살표



벡터: 내적(Dot product)

- ✓ 벡터 간의 유사도 계산: Scalar 값



$$\underbrace{\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ \vdots \\ v_n \end{bmatrix} \cdot \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ \vdots \\ w_n \end{bmatrix}}_{\text{Dot product}} = \begin{matrix} v_1 w_1 \\ + \\ v_2 w_2 \\ + \\ v_3 w_3 \\ + \\ \vdots \\ + \\ v_n w_n \end{matrix} \parallel -4.04$$

- 반대방향이면 -
- 같은 방향이면 +
- 직교하면 ?

벡터: 내적(Dot product)

✓ 유사도 계산



		Tech	Fruitiness	
Sim		1	4	Tech Fruitiness
		0	3	
Sim		1	4	Tech Fruitiness
		3	0	
Sim		0	3	Tech Fruitiness
		3	0	

$$1 \cdot 0 + 4 \cdot 3 = 12$$

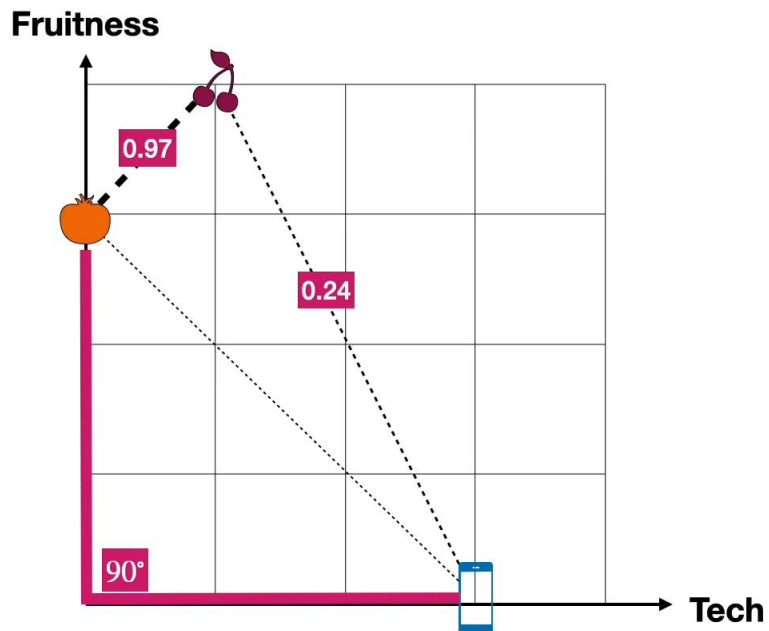
$$1 \cdot 3 + 4 \cdot 0 = 3$$

$$0 \cdot 3 + 3 \cdot 0 = 0$$

https://www.youtube.com/watch?v=UPtG_38Oq

벡터: Cosine similarity

✓ 유사도 계산



Sim  $\cos(14^\circ) = 0.97$



Sim  $\cos(76^\circ) = 0.24$

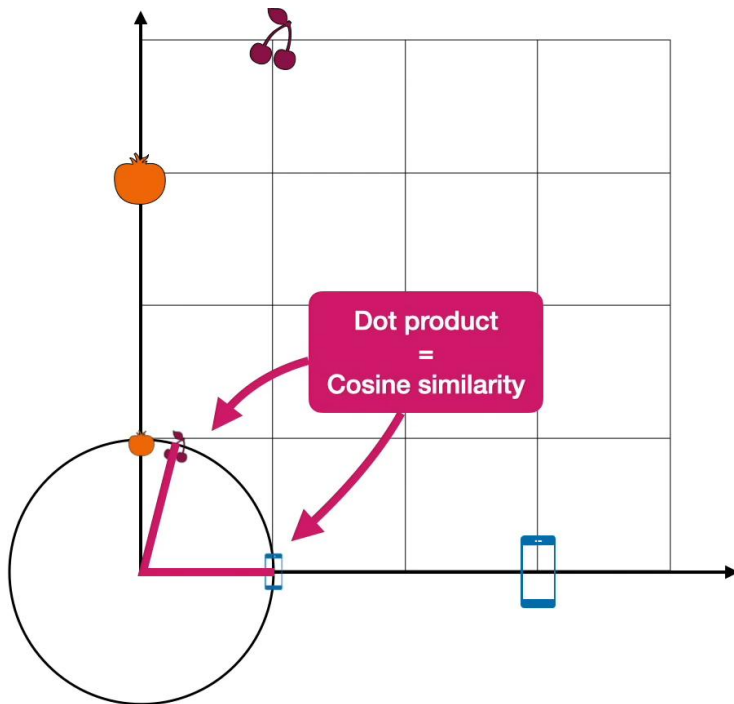


Sim  $\cos(90^\circ) = 0$



벡터: 내적(Dot product)

- ✓ 단위 벡터에서는 Dot Product와 Cosine similarity는 동일함



✓ 요소별 곱셈

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 \times 1 & 2 \times 0 \\ 3 \times 0 & 4 \times 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 4 \end{bmatrix}$$

✓ 요소별 덧셈

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} = \begin{bmatrix} 11 & 22 \\ 33 & 44 \end{bmatrix}$$

✓ 행렬 곱셈

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} @ \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} (1 \times 1 + 2 \times 0) & (1 \times 0 + 2 \times 1) \\ (3 \times 1 + 4 \times 0) & (3 \times 0 + 4 \times 1) \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

✓ Transpose

$$A = \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix}_{2 \times 3}$$

$$A^T = \begin{bmatrix} a & d \\ b & e \\ c & f \end{bmatrix}_{3 \times 2}$$

선형함수

- ✓ Layer를 통과한 값을 계산

y 실제값

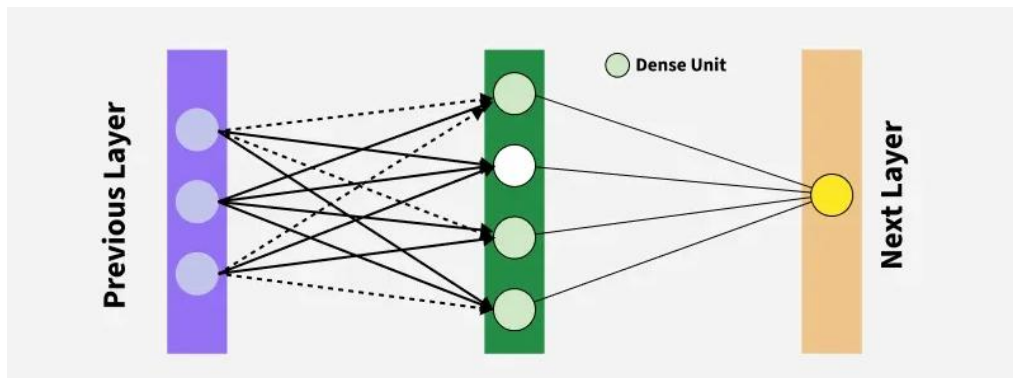
$\hat{y}$ 예측값 (y hat)

$\bar{y}$ 평균값 (y bar)

$$y = f(x)$$

$$y = ax + b$$

$$f(x) = ax + b$$

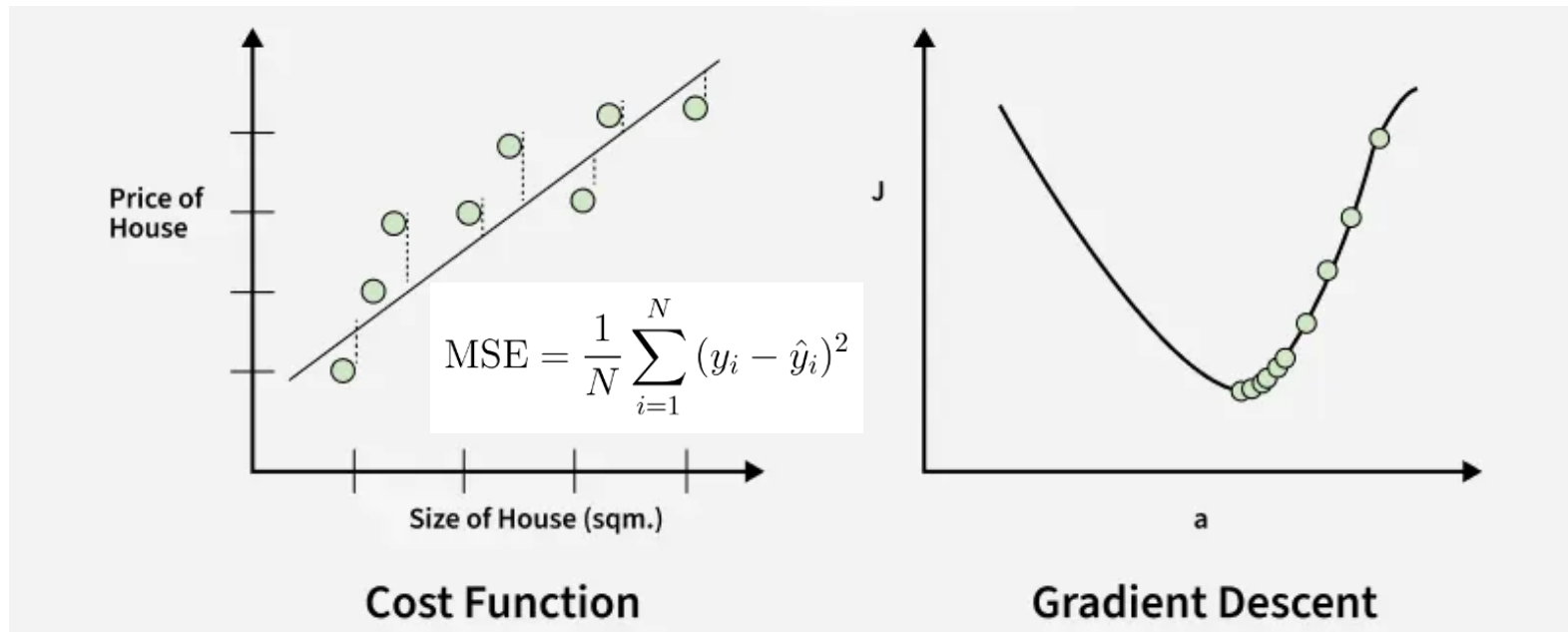


$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

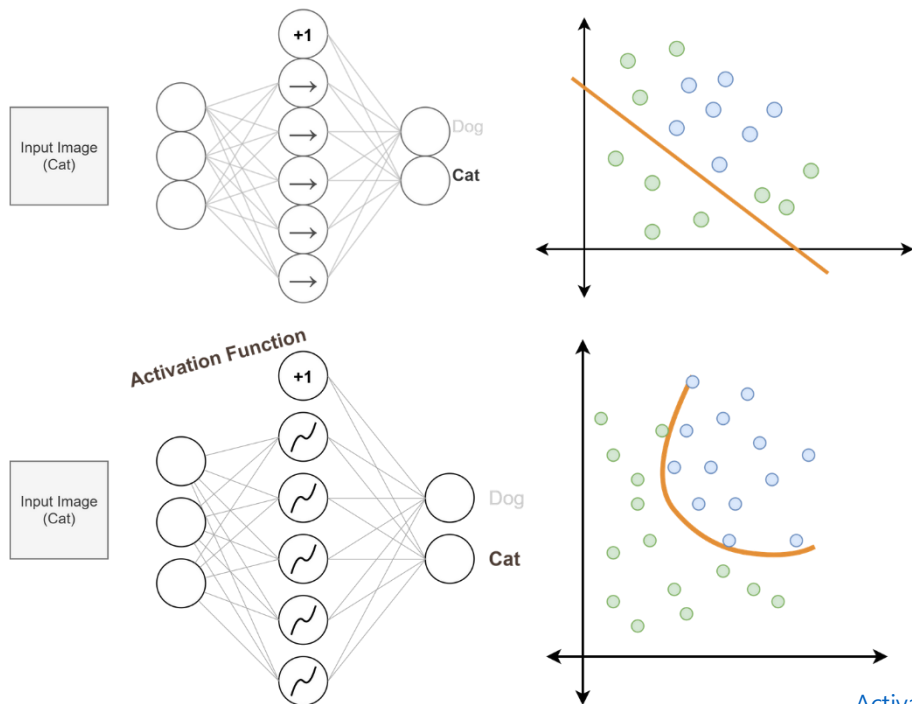
$$z = W^T x + b$$

https://www.youtube.com/watch?v=vS51prw_yfw

- ✓ 실제값과 예측값의 차이를 최소화

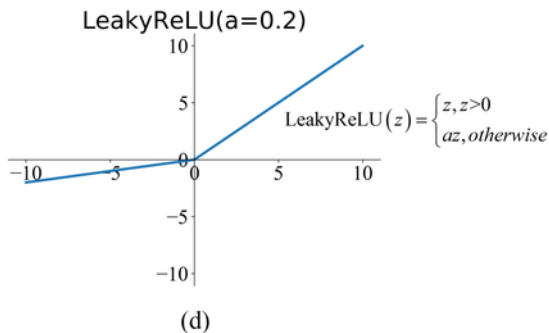
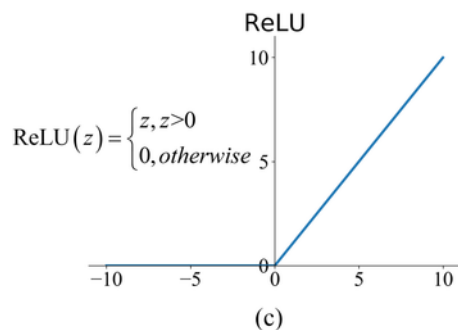
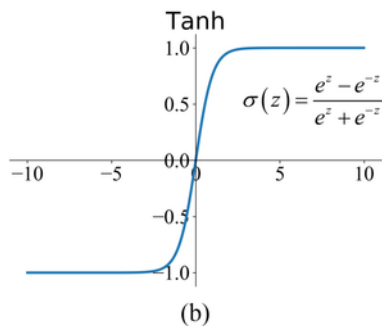
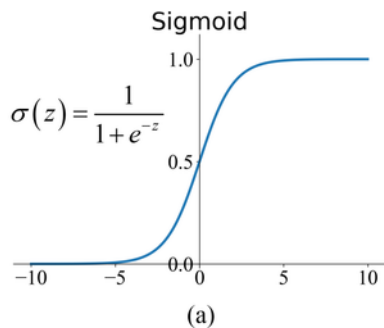


- ✓ 비선형성을 부여하여 복잡한 패턴을 학습이 가능하게 함



[Activation Functions - EXPLAINED!](#)

- ✓ 선형 결합 결과(z)를 비선형적인 신호로 변환하여 다음 층으로 전달



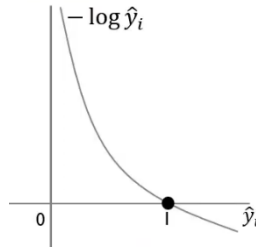
지수함수 로그함수

✓ 지수함수

- 작은 차이를 극적으로 확대(단어 A: 10점, 단어 B: 8점, 단어 C: 1점 -> 단어 A e^{10} :약 22026, 단어 B (e^8): 약 2981, 단어 C (e^1): 약 2.7)
- 모든 숫자를 양수로 변환

✓ 로그함수

- $2^x = 10$ 일 때, 실수 x 의 값은 얼마인가요? $\log_2 10$
- 곱셈을 덧셈으로 변경할 수 있음: 연산속도향상, 0으로 소멸해버리는 '언더플로우' 방지
 $\log(A \times B) = \log A + \log B$ $\log(A/B) = \log A - \log B$
- 틀렸을 때 강력한 피드백을 줌
- 자연로그($\ln x$)를 미분하면 아주 깔끔하게 $1/x$ 가 됩니다.



✓ 자연상수(e)

$$\log_e(x) = \ln(x)$$

- 자연 상수를 지수나 로그 함수에 사용: e^x (모든 지수함수는 자연상수로 표현 가능),
- 미분했을 때 자기 자신이 나옴
 - 지수함수(e^x)를 미분하면 그대로 e^x 가 됩니다.

Softmax와 Sigmoid

✓ Softmax

- 여러 출력값 $z_1, z_2, \dots, z_K$ 을 각각의 확률 $P_1, P_2, \dots, P_K$ 로 변환하며, 모든 확률의 합은 1이 됨, $P_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$
- 다중 분류, 상호 의존성을 가진 활성화 함수에 사용

✓ Sigmoid

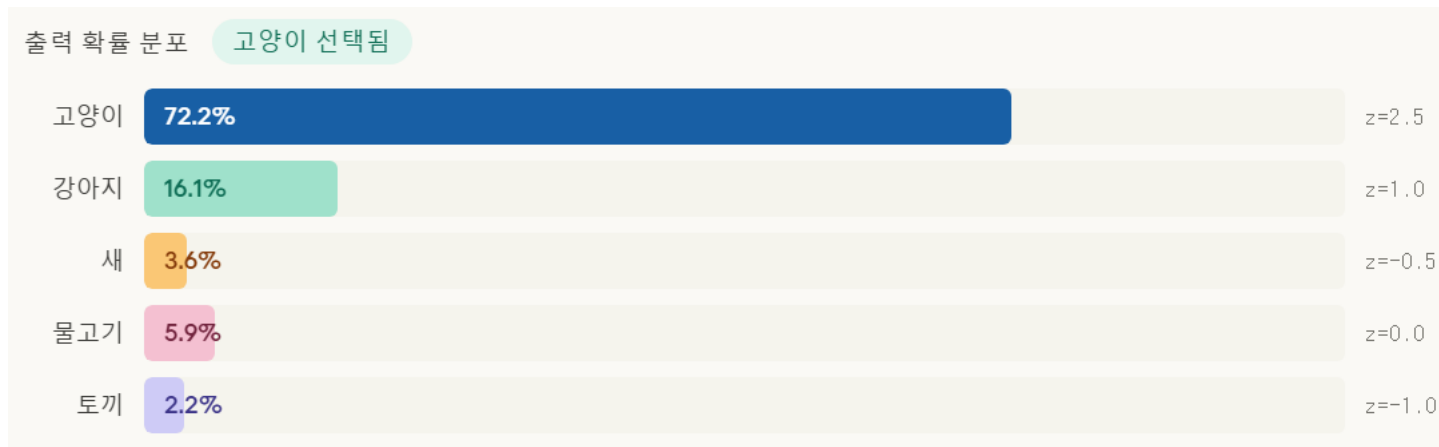
- 입력값 z 를 0과 1 사이의 확률로 변환, $\sigma(z) = \frac{1}{1+e^{-z}}$
- 이진 분류, 활성화 함수에 사용

✓ 클래스가 2개($K = 2$)인 Softmax 함수 = Sigmoid 함수

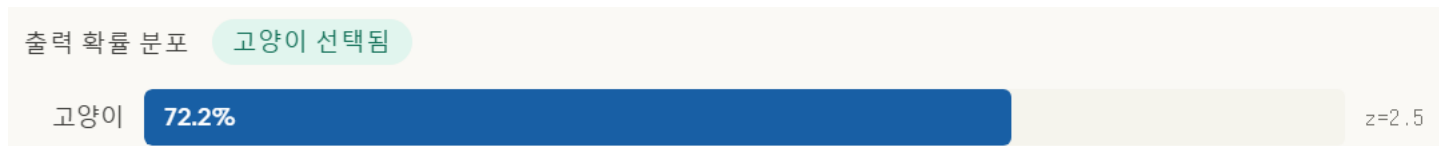
- $P_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}}$
- $P_1 = \frac{1}{1 + \frac{e^{z_2}}{e^{z_1}}}$
- $P_1 = \frac{1}{1 + e^{-(z_1 - z_2)}}$ ($z = z_1 - z_2$ 로 두면 동일)

Softmax와 Sigmoid

✓ Softmax

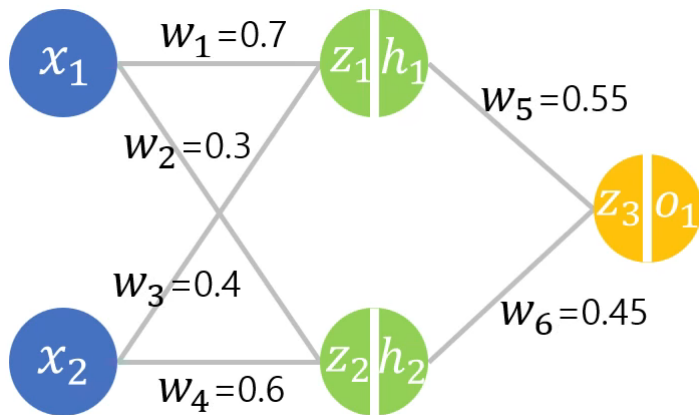


✓ Sigmoid



✓ 3단계로 진행해서 가중치를 업데이트함

- 순전파
- Loss 계산
- 역전파



Feedforward, Loss 계산

✓ 은닉층 계산

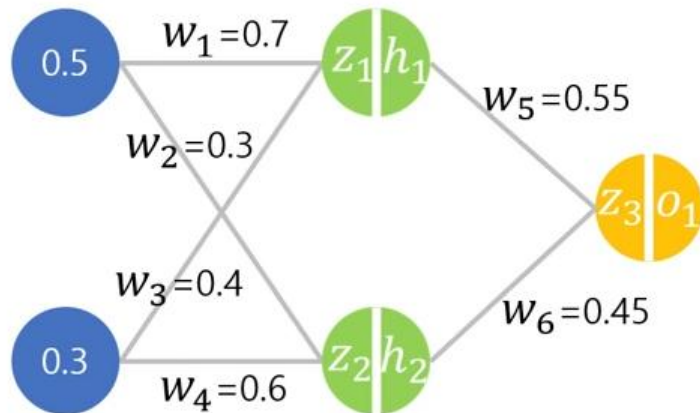
- $z_1 = (x_1 \cdot w_1) + (x_2 \cdot w_3) = (0.5 \cdot 0.7) + (0.3 \cdot 0.4) = \mathbf{0.47}$
- $h_1 = \sigma(z_1) = \text{sigmoid}(0.47) = \mathbf{0.615}$
- $z_2 = (x_1 \cdot w_2) + (x_2 \cdot w_4) = (0.5 \cdot 0.3) + (0.3 \cdot 0.6) = \mathbf{0.33}$
- $h_2 = \sigma(z_2) = \text{sigmoid}(0.33) = \mathbf{0.582}$

✓ 출력층 계산

- $z_3 = (h_1 \cdot w_5) + (h_2 \cdot w_6) = (0.615 \cdot 0.55) + (0.582 \cdot 0.45) \approx \mathbf{0.6}$
- $o_1 = \sigma(z_3) = \text{sigmoid}(0.6) = \mathbf{0.645}$

✓ Loss 계산: 실제값을 1이라 가정하면

- $C = \frac{1}{n} \sum_{i=1}^n (y_i - a_i)^2$
- $C = \frac{1}{1} \sum_1^1 (1 - 0.645)^2 = \mathbf{0.126025}$



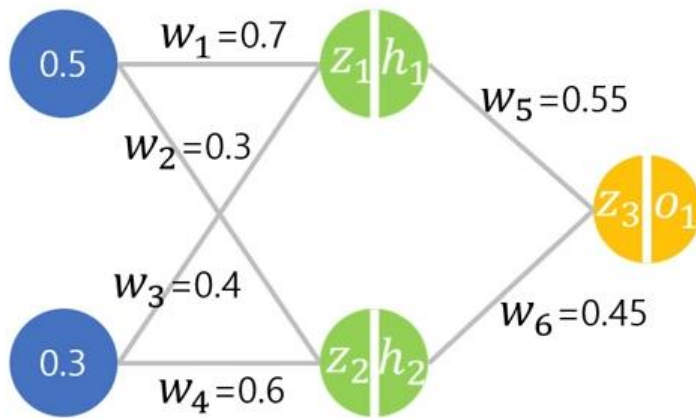
Backprobagation: 가중치 w_5, w_6 계산

✓ w_5 기울기 계산

- $\frac{\partial C}{\partial w_5} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5} = -0.71 \cdot 0.229 \cdot 0.615 \approx -0.01$
- $\frac{\partial C}{\partial o_1} = -2(y - o_1) = -2(1 - 0.645) = -0.71$
- $\frac{\partial o_1}{\partial z_3} = o_1(1 - o_1) = 0.645(1 - 0.645) \approx 0.229$
- $\frac{\partial z_3}{\partial w_5} = h_1 = 0.615$ ($z_3 = h_1 w_5 + h_2 w_6$)

✓ w_5 가중치 업데이트

- $w_5^{new} = w_5 - \eta \cdot \frac{\partial C}{\partial w_5} = 0.55 - 0.1 \cdot (-0.01) = 0.551$



$$\frac{\partial C}{\partial w_5} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5}$$

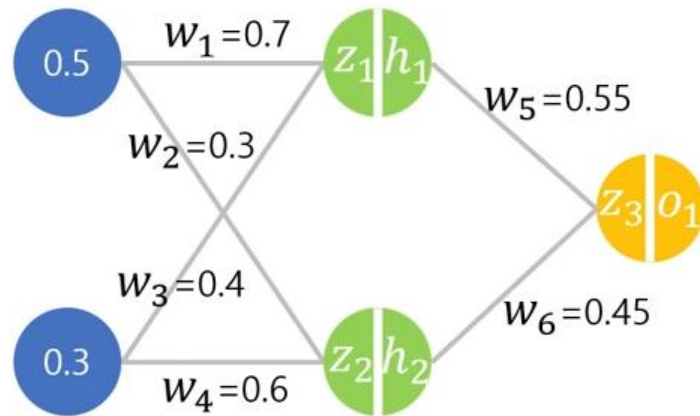
Backprobagation: 가중치 w_5, w_6 계산

✓ w_6 기울기 계산

- $\frac{\partial C}{\partial w_6} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_6} = -0.71 \cdot 0.229 \cdot 0.582 \approx -0.095$
- $\frac{\partial z_3}{\partial w_6} = h_2 = 0.582$

✓ w_6 가중치 업데이트

- $0.45 - (0.1 \cdot -0.095) = 0.45 + 0.0095 = 0.4595$



$$\frac{\partial C}{\partial w_6} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_6}$$

✓ 문제

- 치타는 사자보다 2배 빠름
- 사자는 곰보다 2배 빠름
- 곰은 사람보다 1.5배 빠름
- 치타는 사람에 비해 몇배가 빠를까?

✓ 정답: Chain Rule로 계산

$$\frac{d\text{치타}}{d\text{사람}} = \frac{d\text{치타}}{d\text{사자}} \cdot \frac{d\text{사자}}{d\text{곰}} \cdot \frac{d\text{곰}}{d\text{사람}} = 2 \times 2 \times 1.5 = 6$$

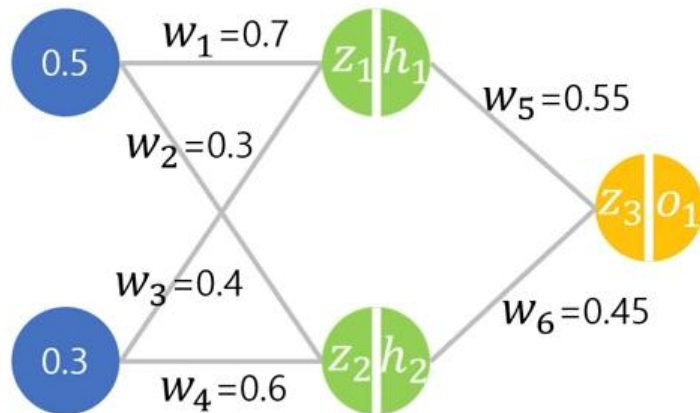
Backprobagation: 가중치 w_1 계산

✓ w_1 기울기 계산

- $\frac{\partial C}{\partial w_1} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1} = (-0.16259) \cdot 0.55 \cdot 0.237 \cdot 0.5 \approx -0.0106$
- $\frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} = -0.71 \cdot 0.229 = -0.16259$
- $\frac{\partial z_3}{\partial h_1} = w_5 = 0.55$
- $\frac{\partial h_1}{\partial z_1} = h_1(1 - h_1) = 0.615(1 - 0.615) \approx 0.237$
- $\frac{\partial z_1}{\partial w_1} = x_1 = 0.5$

✓ w_1 가중치 업데이트

- $w_1^{new} = w_1^{old} - (0.1 \cdot -0.0106) = 0.7 + 0.00106 = 0.7010$



$$\frac{\partial C}{\partial w_1} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

Loss 계산

✓ 은닉층 계산

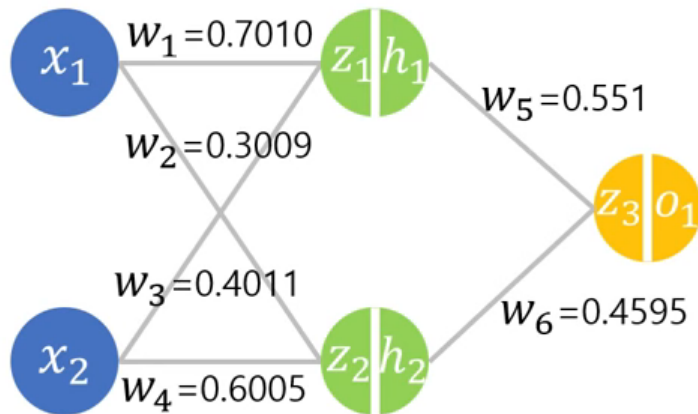
- $z_1 = (0.5 \cdot 0.7010) + (0.3 \cdot 0.4011) = \mathbf{0.4708}$
- $h_1 = \sigma(0.4708) = \mathbf{0.6156}$
- $z_2 = (0.5 \cdot 0.3009) + (0.3 \cdot 0.6005) = \mathbf{0.3306}$
- $h_2 = \sigma(0.3306) = \mathbf{0.5819}$

✓ 출력층 계산

- $z_3 = (0.6156 \cdot 0.551) + (0.5819 \cdot 0.4595) = \mathbf{0.6066}$
- $o_1 = \sigma(0.6066) = \mathbf{0.6472}$

✓ Loss 계산

- $C = (1 - 0.6472)^2 = (0.3528)^2 = \mathbf{0.1245}$
- $\mathbf{0.1245 < 0.1260}$



경사하강법(Gradient Descent)

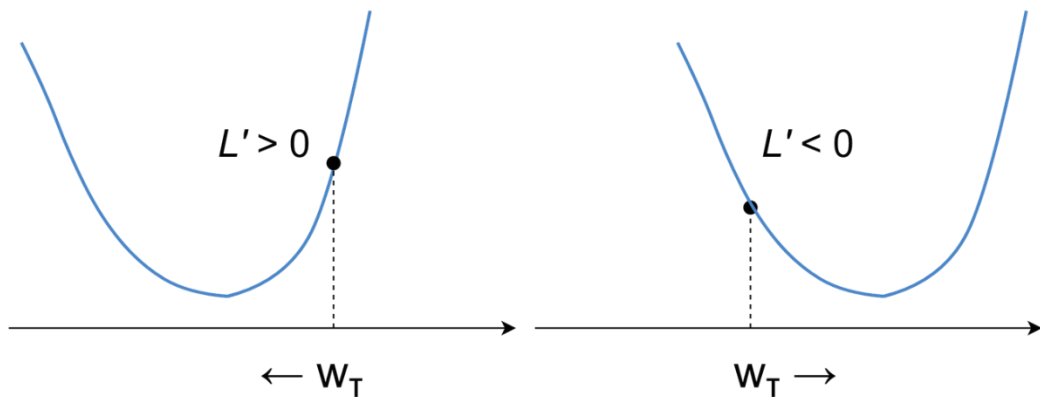
✓ Loss를 최소화하기 위해 가중치를 찾는 과정

- Optimizer가 학습 과정에서 스스로 조율해 나감

$$w_{T+1} = w_T - \alpha \cdot L'(w_T)$$

$0 < \alpha < 1$: learning rate

좀 더 섬세하고 촘촘히? **Small α**
좀 더 빠르게? **Large α**



[Neural Network 9] 역전파
backpropagation 알고리즘
(수정본)

KL Divergence, Cross Entropy

- ✓ 5변을 가진 주사위를 굴려서 나올 확률

5 sided die

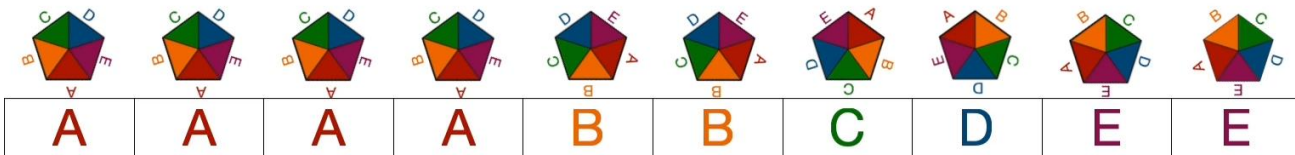
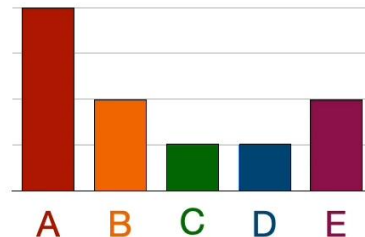
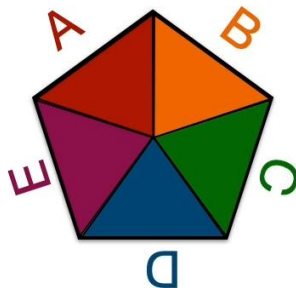
$$p(A) = 0.4$$

$$p(B) = 0.2$$

$$p(C) = 0.1$$

$$p(D) = 0.1$$

$$p(E) = 0.2$$

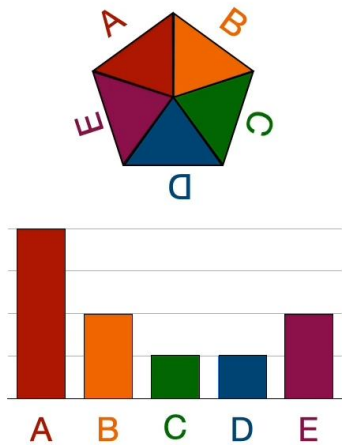


Cross Entropy

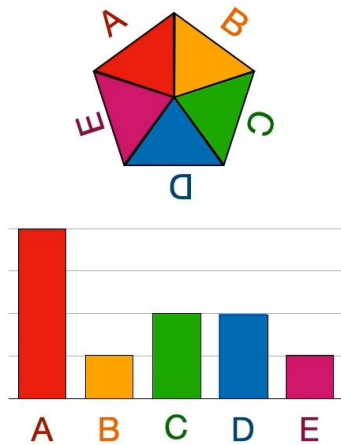
- 어느 확률이 주어진 시퀀스를 재현한 것일까?

A	A	A	A	B	B	C	D	E	E
---	---	---	---	---	---	---	---	---	---

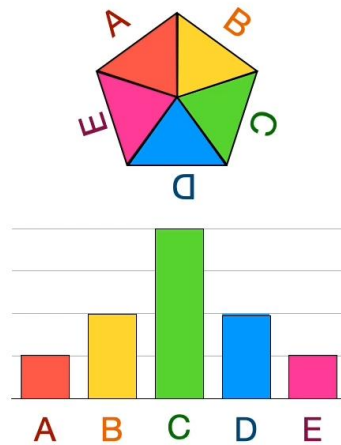
Die 1 (original)



Die 2



Die 3



Cross Entropy

- Die 10 Sequence를 가질 확률

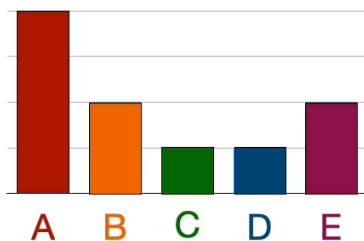
A	A	A	A	B	B	C	D	E	E
---	---	---	---	---	---	---	---	---	---

$$\log(0.4 \cdot 0.4 \cdot 0.4 \cdot 0.4 \cdot 0.2 \cdot 0.2 \cdot 0.1 \cdot 0.1 \cdot 0.2 \cdot 0.2) = -14.71$$

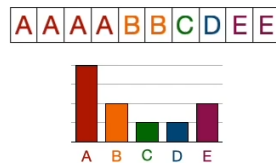
$$\log(0.4) + \log(0.4) + \log(0.4) + \log(0.4) + \log(0.2) + \log(0.2) + \log(0.1) + \log(0.1) + \log(0.2) + \log(0.2)$$

$$-\frac{4 \log(0.4) + 2 \log(0.2) + 1 \log(0.1) + 1 \log(0.1) + 2 \log(0.2)}{10} = 1.471$$

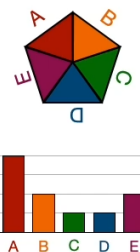
Die 1



$$\begin{aligned} p(A) &= 0.4 \\ p(B) &= 0.2 \\ p(C) &= 0.1 \\ p(D) &= 0.1 \\ p(E) &= 0.2 \end{aligned}$$



$$\begin{aligned} & -0.4 \log(0.4) \\ & -0.2 \log(0.2) \\ & -0.1 \log(0.1) \\ & -0.1 \log(0.1) \\ & -0.2 \log(0.2) \end{aligned}$$



$$-\sum_i p_i \log(p_i) = 1.471$$

Cross Entropy

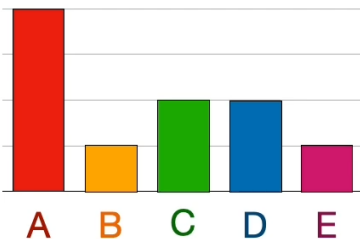
- Die2가 Sequence를 가질 확률

A	A	A	A	B	B	C	D	E	E
0.4	0.4	0.4	0.4	0.1	0.1	0.2	0.2	0.1	0.1

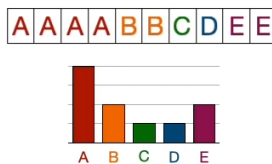
= 0.0000001024

$$-\frac{4\log(0.4) + 2\log(0.1) + 1\log(0.2) + 1\log(0.2) + 2\log(0.1)}{10} = 1.609$$

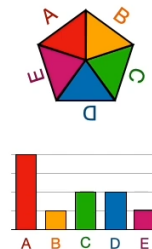
Die 2



$$\begin{aligned} q(A) &= 0.4 \\ q(B) &= 0.1 \\ q(C) &= 0.2 \\ q(D) &= 0.2 \\ q(E) &= 0.1 \end{aligned}$$



$$\begin{aligned} &P \\ &Q \\ - &0.4\log(0.4) \\ - &0.2\log(0.1) \\ - &0.1\log(0.2) \\ - &0.1\log(0.2) \\ - &0.2\log(0.1) \end{aligned}$$



Cross entropy $H(P|Q) = -\sum_i p_i \log(q_i) = 1.609$

Cross Entropy

- Die3가 Sequence를 가질 확률

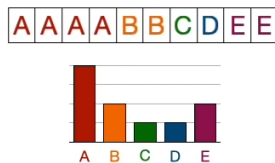
A	A	A	A	B	B	C	D	E	E
0.1	0.1	0.1	0.1	0.2	0.2	0.4	0.2	0.1	0.1

= 0.0000000032

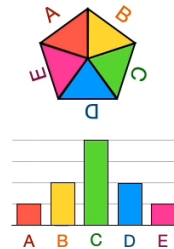
$$- \frac{4 \log(0.1) + 2 \log(0.2) + 1 \log(0.4) + 1 \log(0.2) + 2 \log(0.1)}{10} = 1.956$$



$$\begin{aligned} r(A) &= 0.1 \\ r(B) &= 0.2 \\ r(C) &= 0.4 \\ r(D) &= 0.2 \\ r(E) &= 0.1 \end{aligned}$$



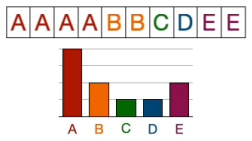
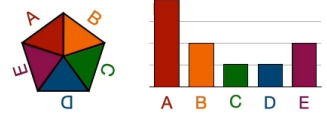
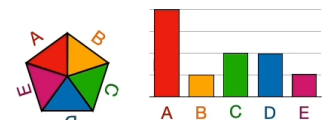
$$\begin{aligned} &P \\ &R \\ &- 0.4 \log(0.1) \\ &- 0.2 \log(0.2) \\ &- 0.1 \log(0.4) \\ &- 0.1 \log(0.2) \\ &- 0.2 \log(0.1) \end{aligned}$$



Cross entropy $H(P|R) = - \sum_i p_i \log(r_i) = 1.956$

KL-Divergence

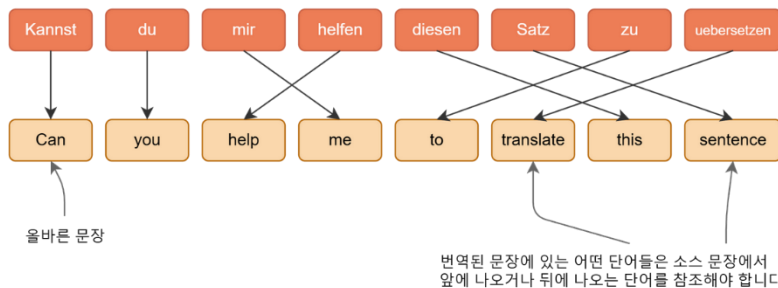
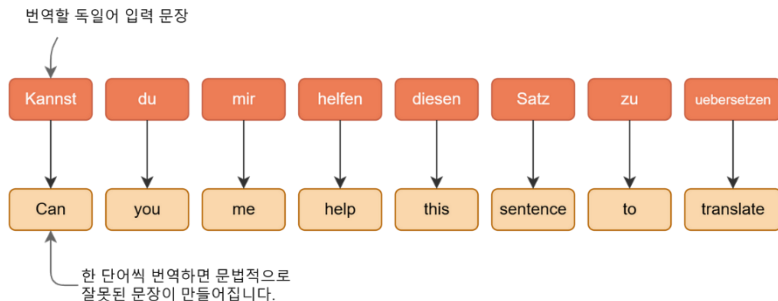
- ✓ Cross Entropy에서 실제 데이터 자체의 Entropy를 뺀 값 (모델이 틀린 만큼의 벌점)

Sequence	Probability	Cross-entropy	KL-divergence
			$D_{KL}(P Q) = \text{Cross Entropy}(P, Q) - \text{Entropy}(P)$
Die 1 	$0.4^4 \cdot 0.1^2 \cdot 0.2^1 \cdot 0.2^1 \cdot 0.1^2$  0.0000004096 	$-0.4 \log(0.4) - 0.2 \log(0.2) - 0.1 \log(0.1) - 0.1 \log(0.1) - 0.2 \log(0.2)$ $H(P) = 1.471$ (entropy)	$D(P P) = 1.471 - 1.471$ 0 
Die 2 	$0.4^4 \cdot 0.1^1 \cdot 0.2^2 \cdot 0.2^2 \cdot 0.1^1$ 0.0000001024 	$-0.4 \log(0.4) - 0.2 \log(0.1) - 0.1 \log(0.2) - 0.1 \log(0.2) - 0.2 \log(0.1)$ $H(P Q) = 1.609$	$D(Q P) = 1.609 - 1.471$ 0.138
Die 3 	$0.4^1 \cdot 0.1^2 \cdot 0.2^4 \cdot 0.2^2 \cdot 0.1^1$ 0.0000000032 	$-0.4 \log(0.1) - 0.2 \log(0.2) - 0.1 \log(0.4) - 0.1 \log(0.2) - 0.2 \log(0.1)$ $H(P R) = 1.956$	$D(R P) = 1.956 - 1.471$ 0.485

언어 모델의 발전

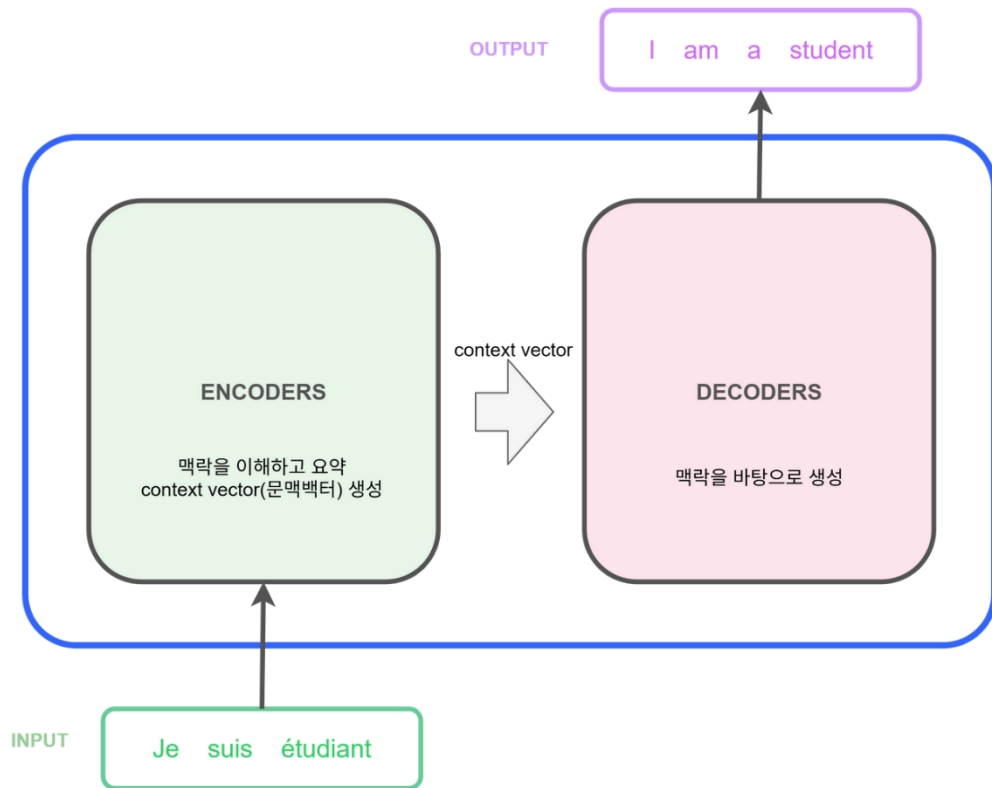
규칙기반 번역

- ✓ 형태소 분석-> 구문 분석 -> 변환 -> 규칙으로 어순 배열



언어모델: Encoders, Decoders

- ✓ 입력을 이해하고, 이해를 바탕으로 생성함

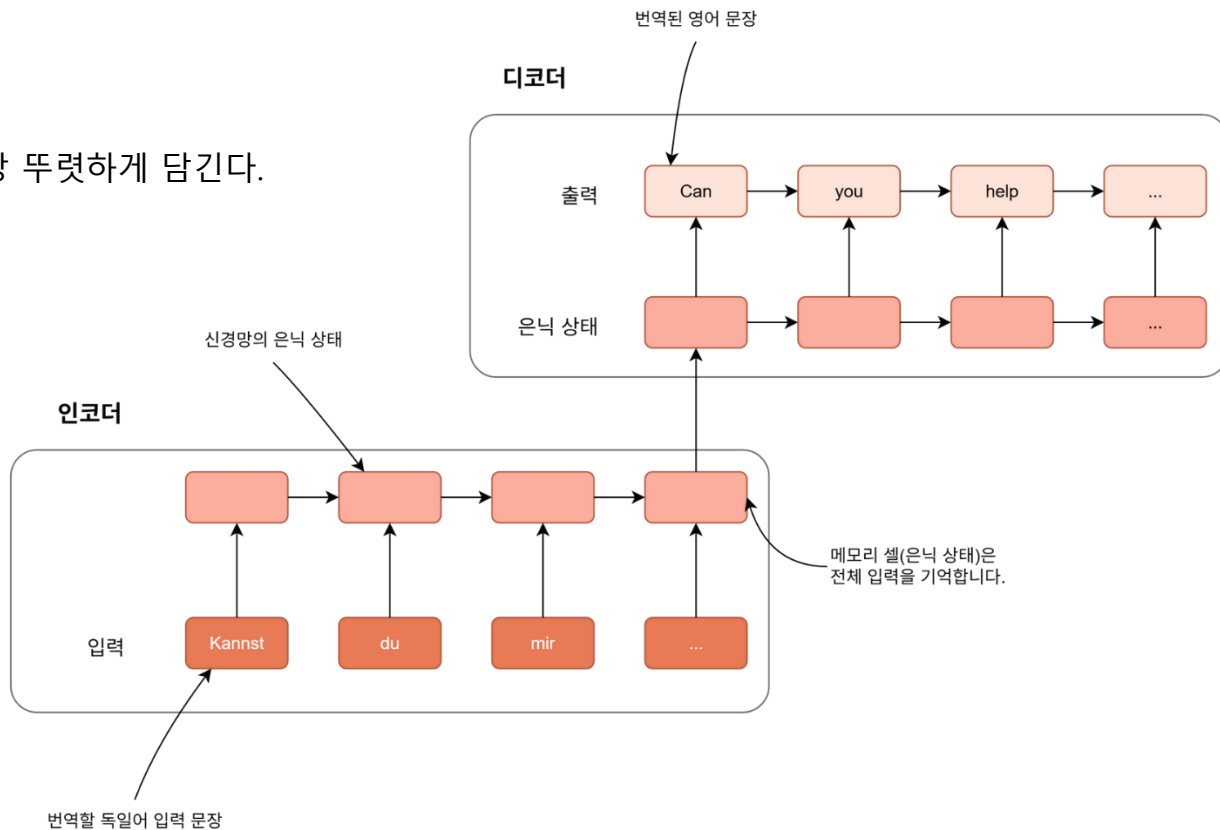
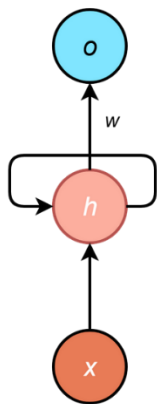


언어모델:RNN

✓ "문맥(Context)"을 기억하는 메모리의 탄생, 가변 길이 처리

✓ 단점

- 길수록 잊혀진다.
- 가장 마지막 정보가 가장 뚜렷하게 담긴다.

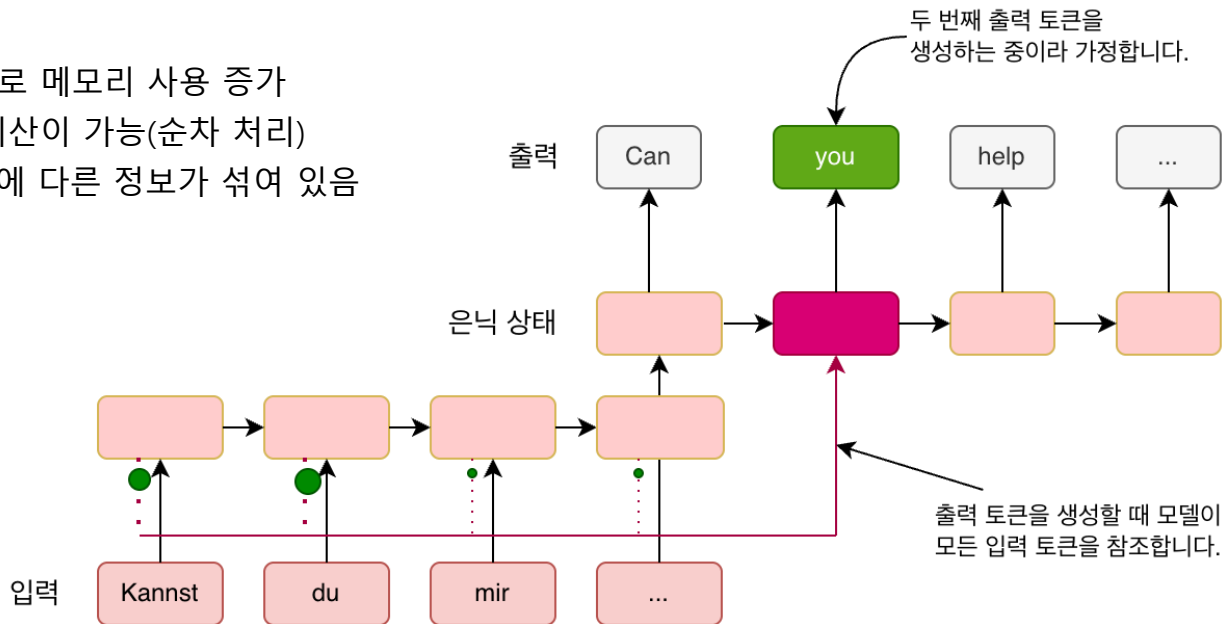


언어모델: RNN Attention

✓ 입력 단어들과의 Attention 참고해서 성능 향상, 어떤 단어를 주목할지를 학습

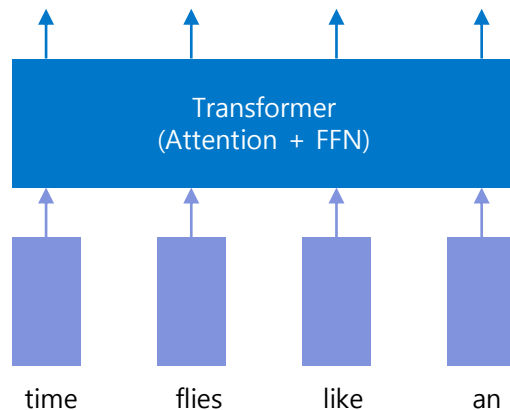
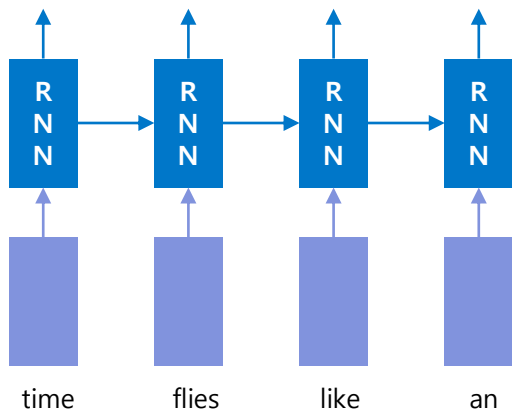
✓ 단점

- 이전 은닉 상태의 저장으로 메모리 사용 증가
- 이전 계산을 해야 다음 계산이 가능(순차 처리)
- 참고하고자 하는 Hidden에 다른 정보가 섞여 있음



언어모델: Transformer

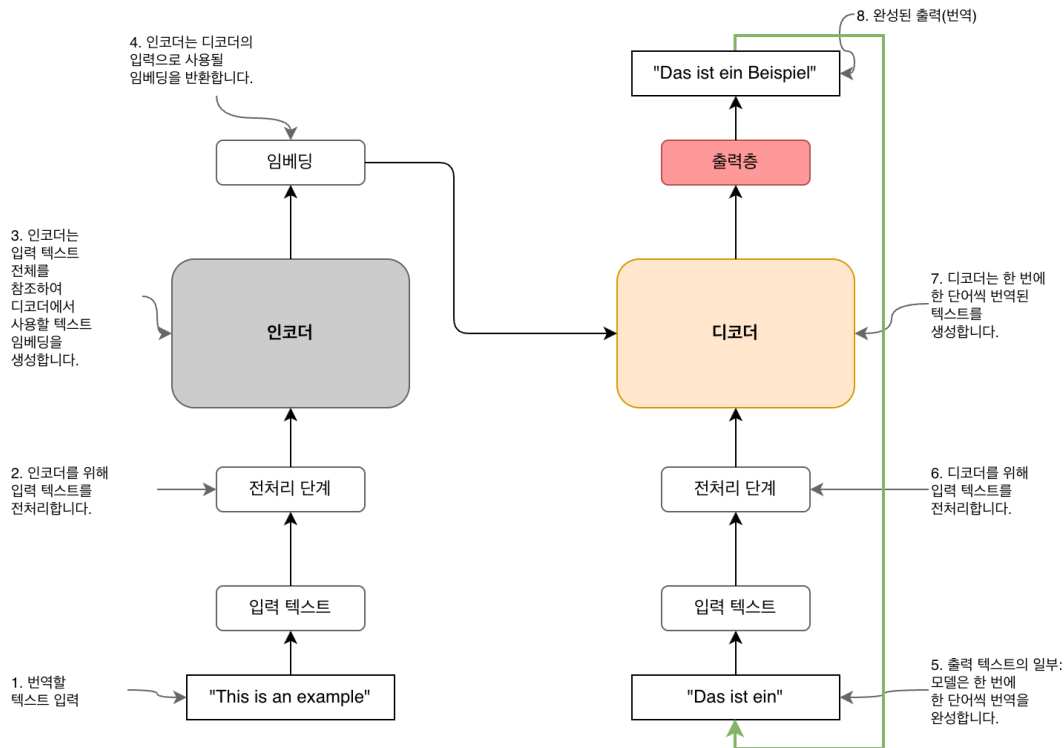
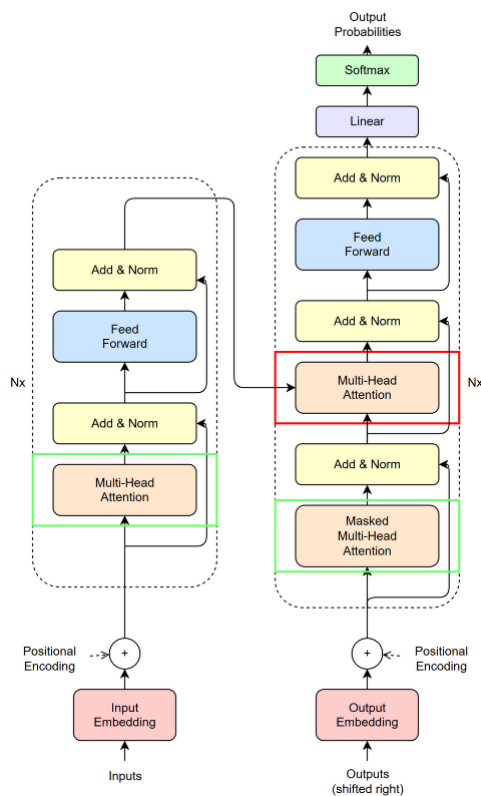
- ✓ 트랜스포머의 핵심은 어텐션(Attention)과 병렬화 가능한 아키텍처



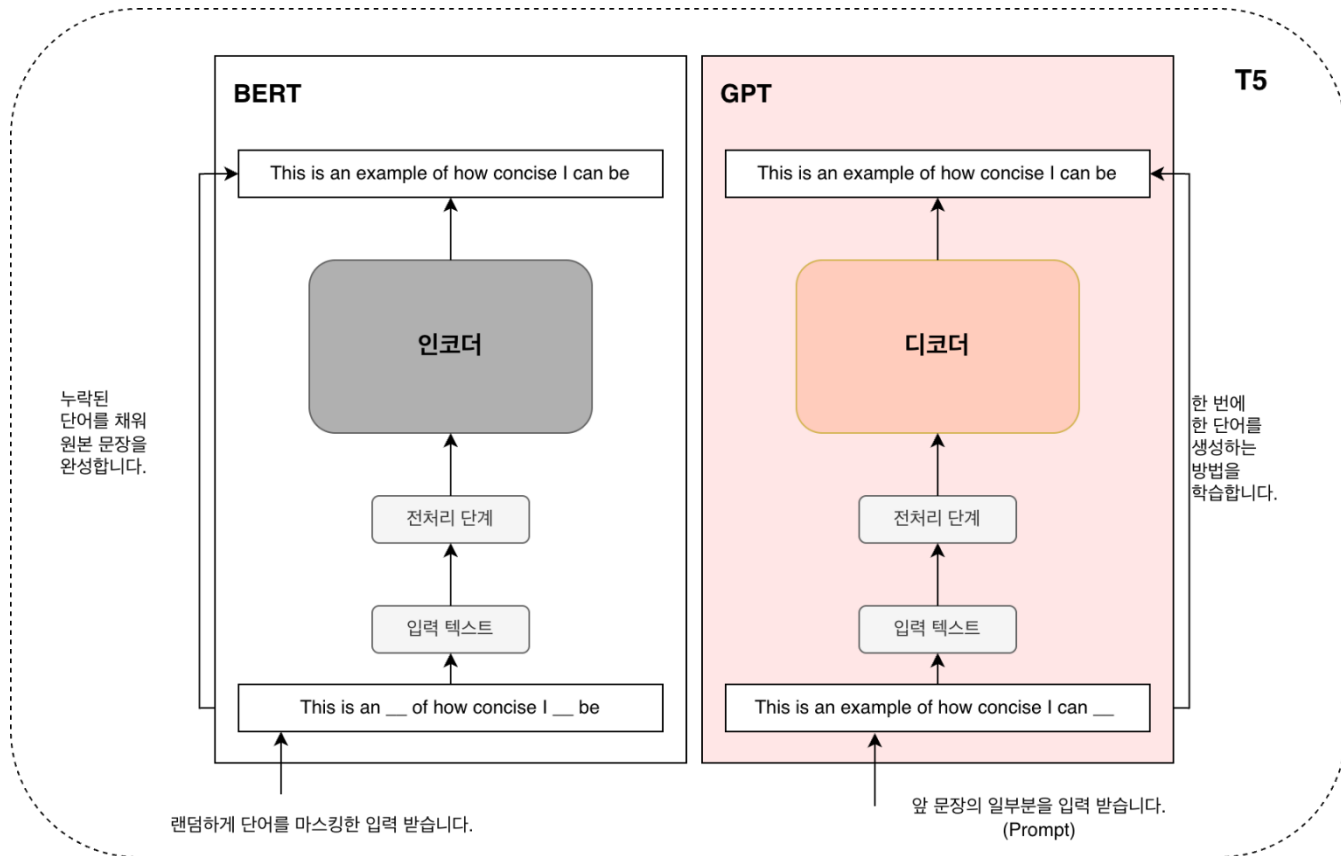
RNN is **hard to parallelize** and not time-efficient !

언어모델: Transformer

✓ Attention과 FFN으로 구성



언어모델: BERT, GPT, T5



✓ Pre-training: [Self-supervised Learning](#)

- BERT,T5: Masked Language Model (MLM), 양방향 학습
- GPT: Causal Language Model (CLM) - Next Word Prediction

BERT, T5

- ◆ 빈칸 채우기 문제
- ◆ 양쪽 문맥 모두 참조 가능

인재개발원은 _____ 에 있습니다

나는 이번 학기에 _____ 강의를 수강합니다

BERT는 트랜스포머의 _____ 구조만을 사용합니다

조성진은 한국이 낳은 훌륭한 _____ 입니다

피보나치 수열은 1, 1, 2, 3, 5, 8, 13, 21, _____ 와 같다

GPT

- ◆ 다음 단어 맞추기 문제
- ◆ 앞쪽 문맥만 참조 가능

인재개발원은 _____

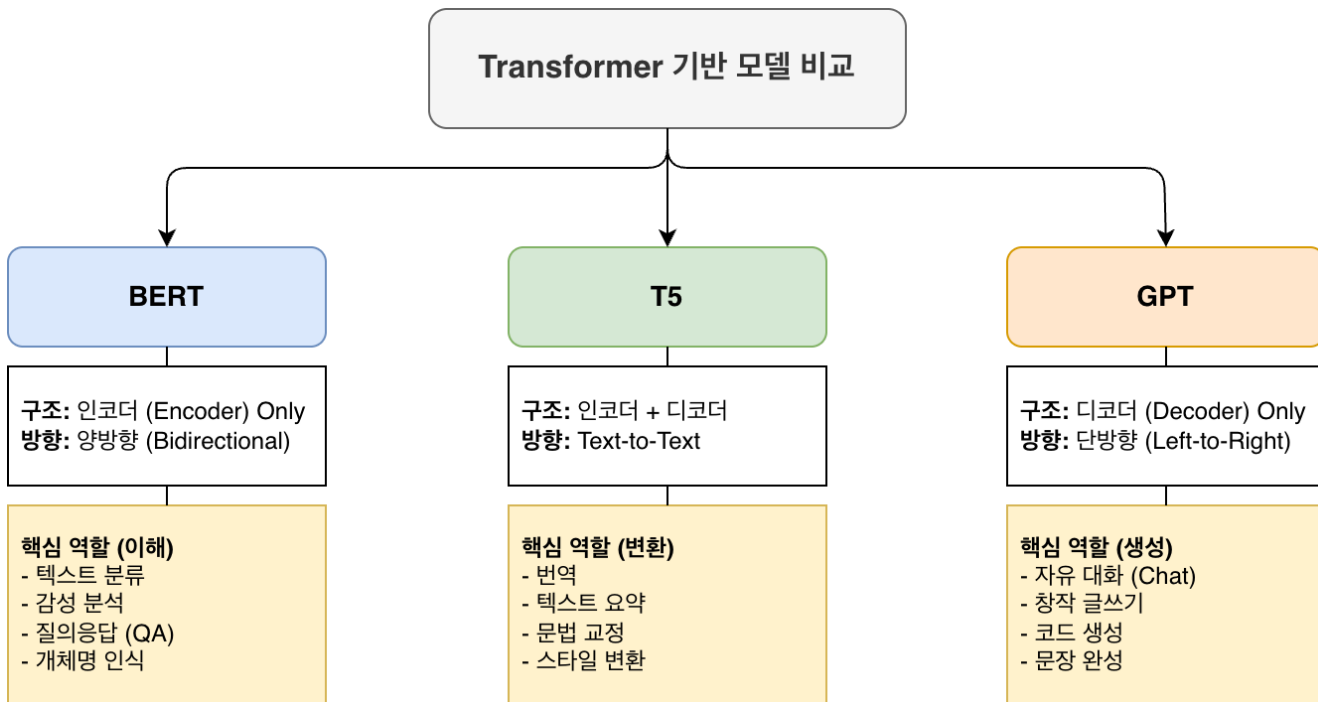
나는 이번 학기에 딥러닝 강의를 _____

GPT는 트랜스포머의 _____

조성진은 한국이 낳은 훌륭한 _____

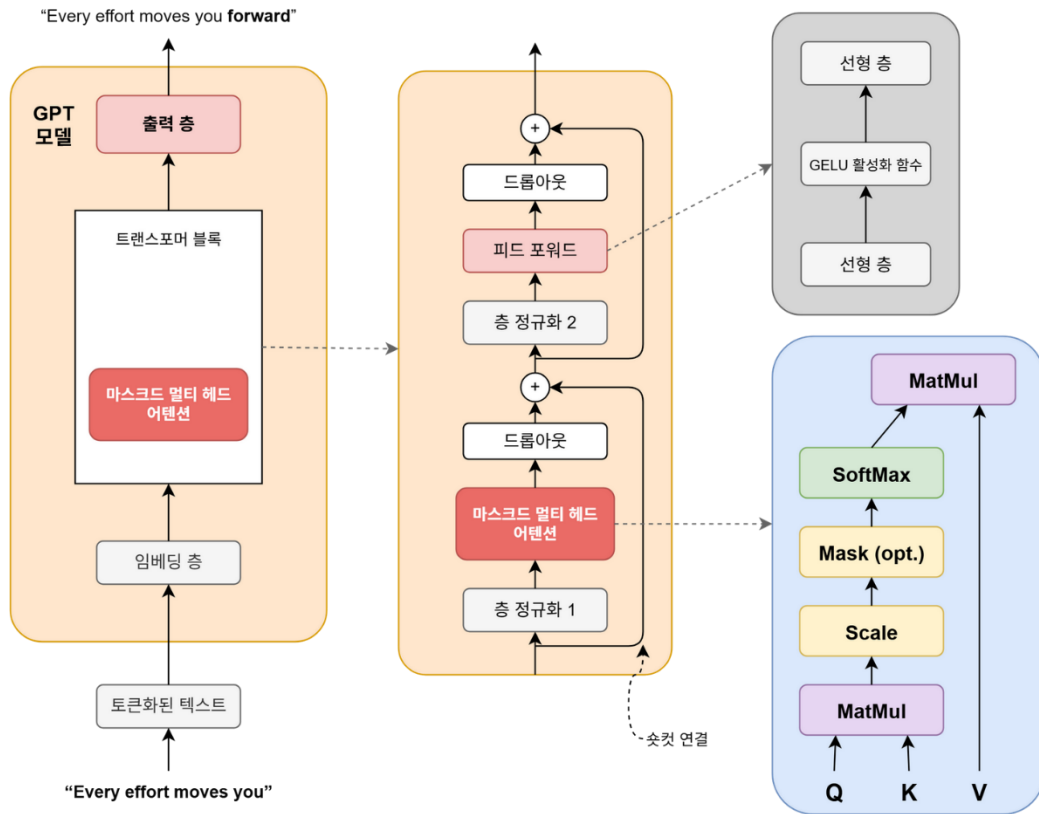
피보나치 수열은 1, 1, 2, 3, 5, 8, 13, 21, _____

언어 모델 비교



언어 모델: GPT

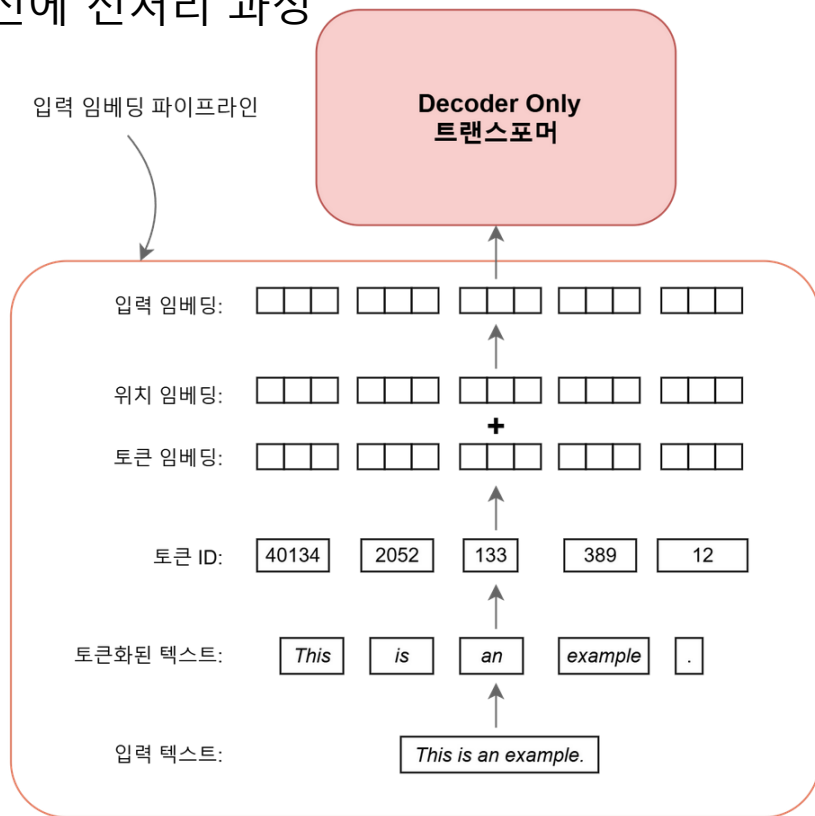
✓ Decoder만 사용



데이터 입력

데이터 입력: 전처리

✓ 모델에 입력하기 전에 전처리 과정



데이터 입력: Tokenizer

- ✓ 입력 문장을 의미 단위로 잘라서 고유 번호를 부여
 - 자주 쓰이는 단어는 가능하면 길게: 데이터 압축을 높일 수 있음
 - 가능하면 의미 단위로 자름

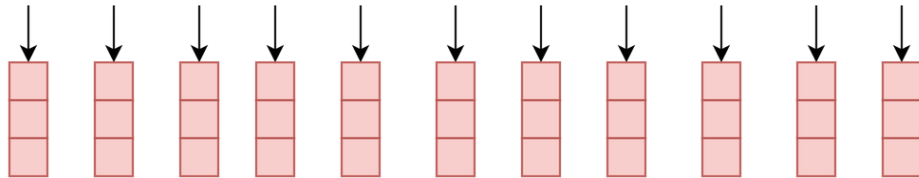
"Today is a beautiful day outside."



["To", "day", "is", "a", "beaut", "iful", "day", "out", "side", "."]

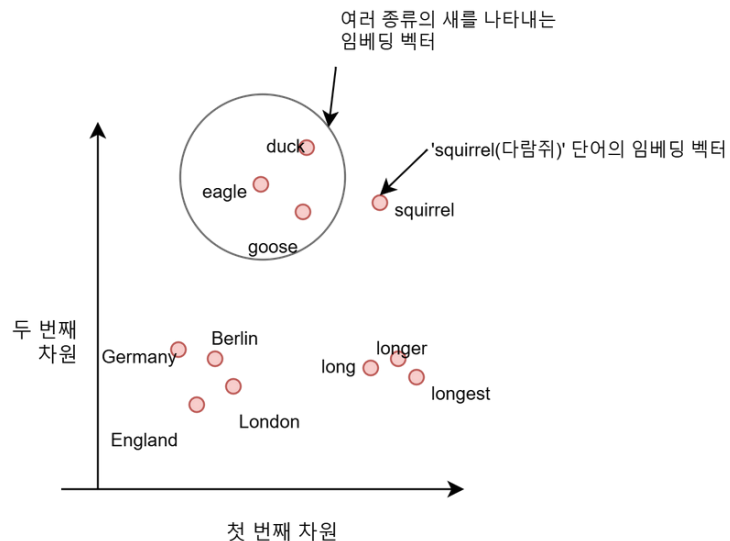
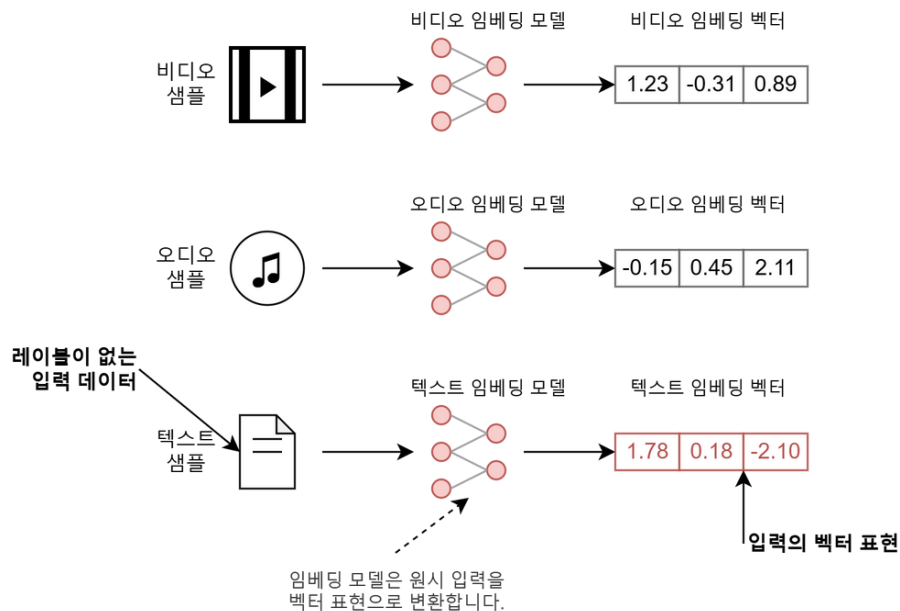


[98, 1452, 43, 15, 2932, 1709, 740, 1452, 3112, 3823, 74]



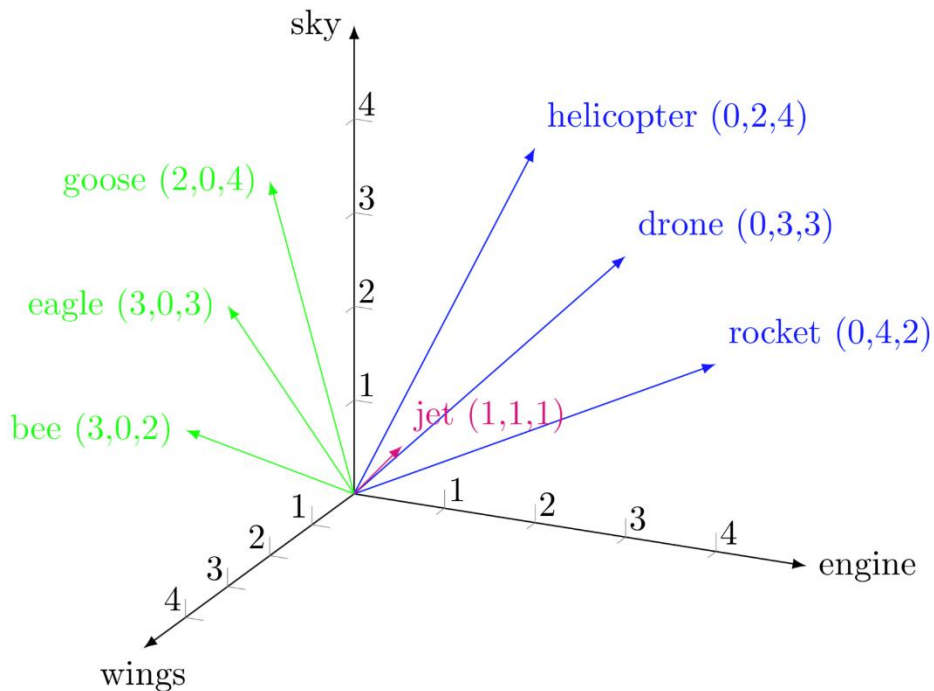
데이터 입력: embedding

✓ 어떤 데이터를 벡터로 표현한 것



데이터 입력: embedding

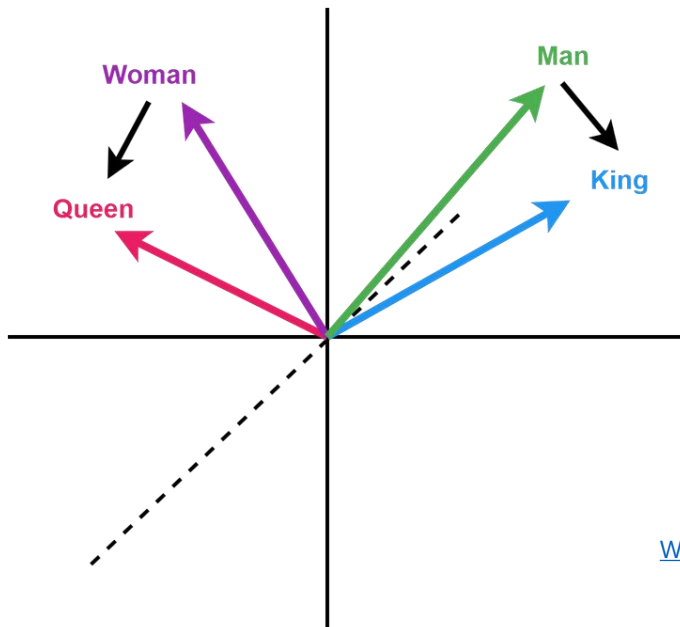
✓ 벡터 공간의 점으로 맵핑



[트랜스포머, ChatGPT가 트랜스포머로 만들어졌죠. - DL5](#)

✓ 비슷한 의미를 가진 단어는 비슷한 공간에 위치

- 단어간 의미를 계산할 수 있음
- $\text{Woman} - \text{Queen} \approx \text{Man} - \text{King}$



[WebVectors: Semantic Calculator](#)

Chapter_1_Exercise_Vector Space.ipynb

[WebVectors: Semantic Calculator](#)

데이터입력: BPE Tokenizer

- 가장 많이 등장하는 쌍 찾아서 추가

deep learning engineer

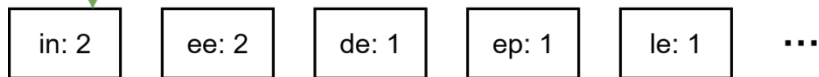


[deep, learning, engineer]

Vocabulary: [d, e, p, l, a, r, n, i, g]



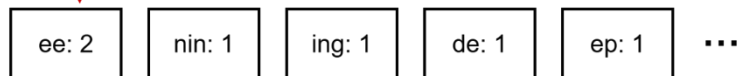
["d e e p", "l e a r n i n g", "e n g i n e e r"]



Vocabulary: [d, e, p, l, a, r, n, i, g, in]



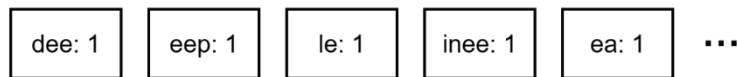
["d e e p", "l e a r n i n g", "e n g i n e e r"]



Vocabulary: [d, e, p, l, a, r, n, i, g, in, ee]

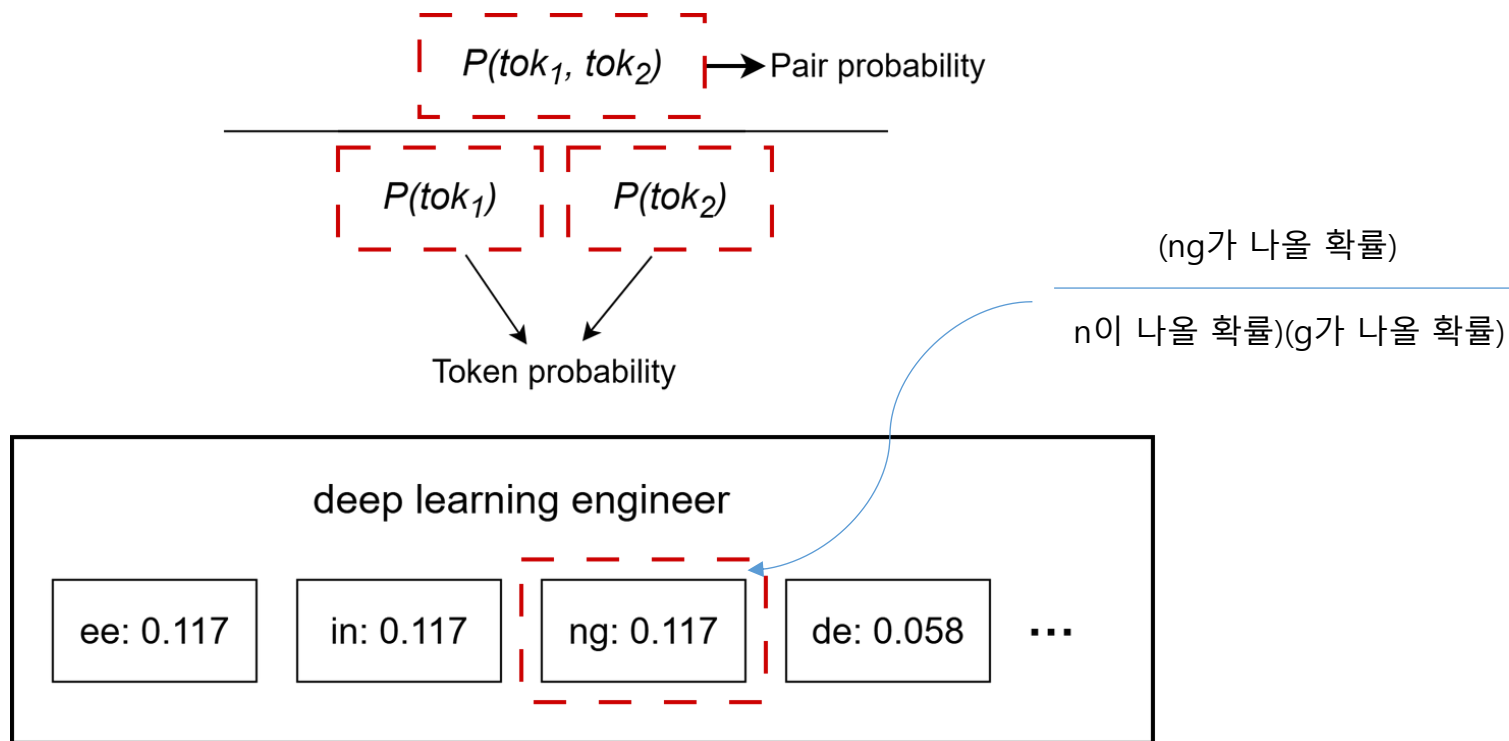


["d e e p", "l e a r n i n g", "e n g i n e e r"]



데이터입력: Wordpiece Tokenizer

- ✓ 각각 나올 확률보다 쌍으로 나올 확률이 높으면 추가



데이터입력: Unigram Tokenizer

✓ 최선의 조합을 찾고, 쓸모없는 것은 버림

("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)

("h", 15) ("u", 36) ("g", 20) ("hu", 15) ("ug", 20) ("p", 17) ("pu", 17) ("n", 16)
("un", 16) ("b", 4) ("bu", 4) ("s", 5) ("hug", 15) ("gs", 5) ("ugs", 5)

$$P(["p", "u", "g"]) = P("p") \times P("u") \times P("g") = 5/210 \times 36/210 \times 20/210 = 0.000389$$

$$P(["pu", "g"]) = P("pu") \times P("g") = 5/210 \times 20/210 = 0.0022676$$

"hug": ["hug"] (score 0.071428)
"pug": ["pu", "g"] (score 0.007710)
"pun": ["pu", "n"] (score 0.006168)
"bun": ["bu", "n"] (score 0.001451)
"hugs": ["hug", "s"] (score 0.001701)

Best 조합

hug는 유지

"hug": ["hug"] (score 0.071428)
"hugs": ["hug", "s"] (score 0.001701)

"hug": ["hu", "g"] (score 0.006802)
"hugs": ["hu", "gs"] (score 0.001701)

pu는 삭제

"pug": ["pu", "g"] (score 0.0022676)
"pun": ["pu", "n"] (score 0.006168)

"pug": ["p", "ug"] (score 0.0022676)
"pun": ["p", "un"] (score 0.006168)

- 확률에 따라 토큰화 결과를 매번 조금씩 다르게 샘플링
playing → play + ing 또는 pl + ay + ing)
- 통계적 특성을 더 잘 반영하는 효율적인 어휘 사전을 구축
- 반면에 속도는 느림

Subword Tokenizer 비교 (BPE vs WordPiece vs Unigram)

BPE (Byte-Pair Encoding)

Bottom-up (상향식)
문자 → 단어

빈도수 (Frequency)
가장 자주 등장하는 쌍 병합

GPT-2, GPT-3, RoBERTa

직관적, 빠른 속도
OOV 해결에 강점

WordPiece

Bottom-up (상향식)
문자 → 단어

우도 (Likelihood)
결합 확률이 높은 쌍 병합

BERT, DistilBERT, Electra

언어 모델 성능 최적화
희귀 단어 관계 파악

Unigram

Top-down (하향식)
전체 → 가지치기(Pruning)

우도 (Likelihood)
손실(Loss)이 적은 것 제거

T5, ALBERT, mBART

Subword Regularization
확률적 토큰화 가능

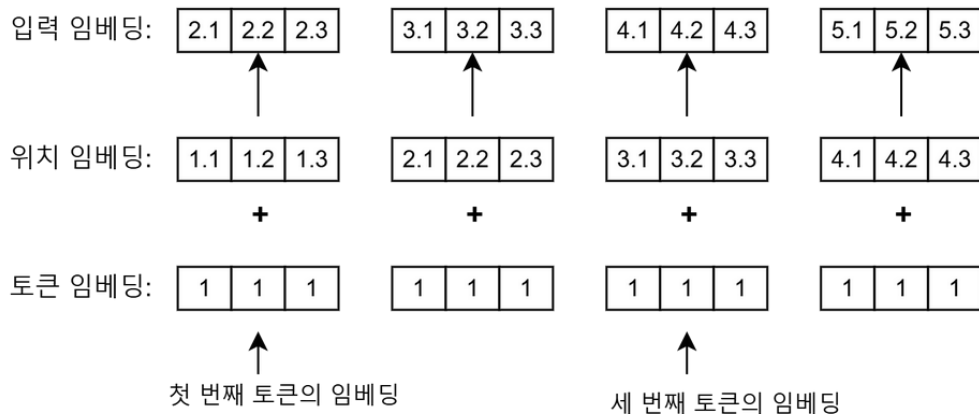
위치 인코딩

✓ 위치 인코딩 필요

- Token에는 단어의 위치 정보가 없음
- Tom ate the meat $\neq$ The meat ate Tom.

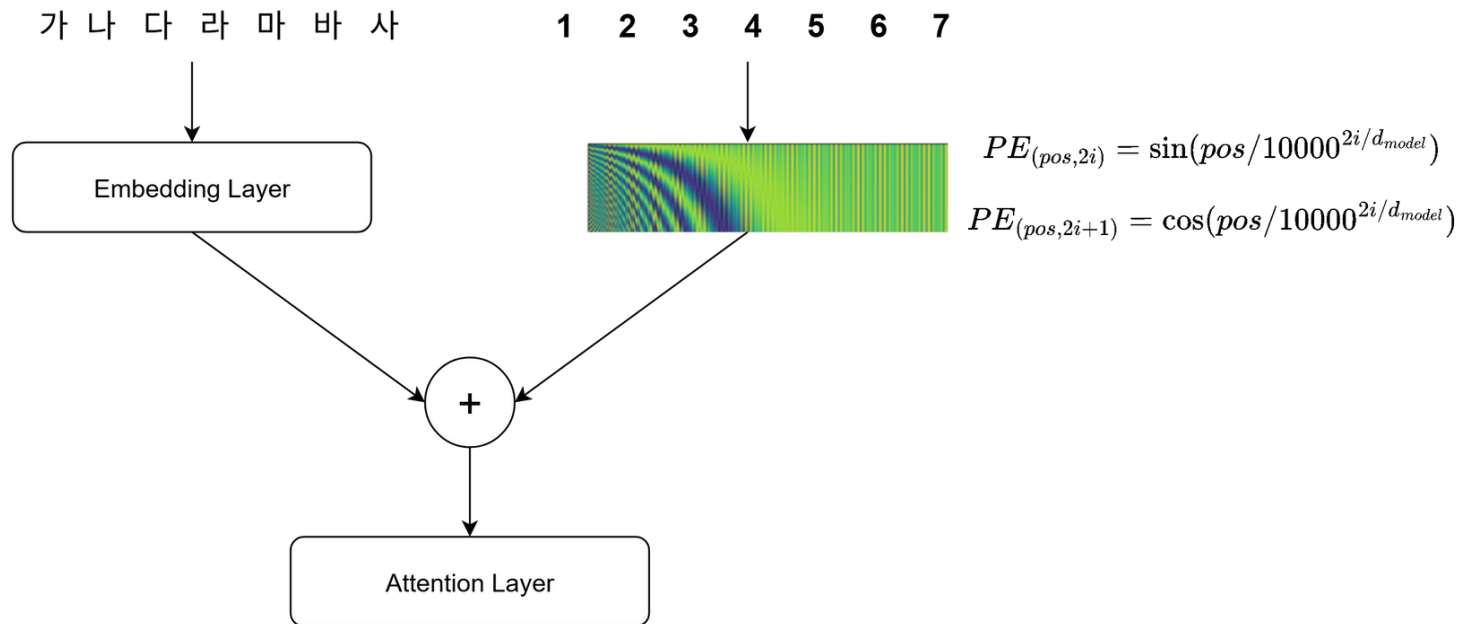
✓ 조건

- 시퀀스 길이에 관계없이 동일한 위치
- 의미적 유사성보다 위치적 유사성이 더 강조되지 않도록 크기가 제한



절대 위치 인코딩

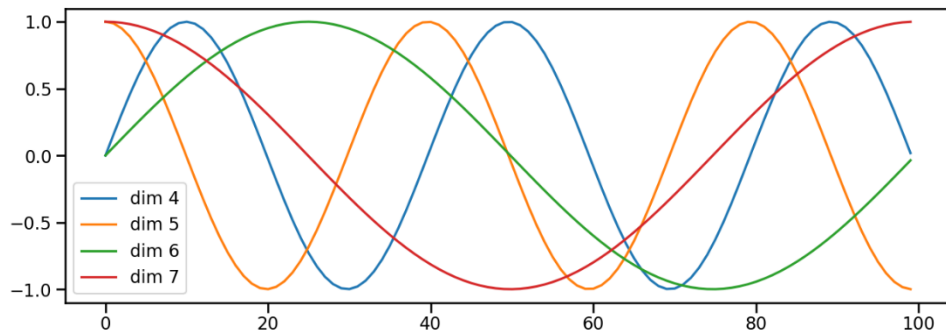
- ✓ 단어의 위치를 절대값으로 표기



Sinusoidal Positional Encoding

✓ 절대 위치: 삼각함수

- 낮은 차원: 주기가 짧아서 빠르게 변함(세밀한 위치 구분)
- 높은 차원: 주기가 매우 길어서 문장이 길어져도 반복되지 않음
- 결과: 수만 단어 길이의 문장에서도 유일한 위치 지문을 만들 수 있음



Position of word Location of embedding vector Size of embedding vector

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

	Position of word	Location of embedding vector	Size of embedding vector
I	1 1 1 1	0 1 0 1	$\sin(0/10000^{(2*0)/4}) = 0$ $\cos(0/10000^{(2*1)/4}) = 1$ $\sin(0/10000^{(2*2)/4}) = 0$ $\cos(0/10000^{(2*3)/4}) = 1$
am	1 1 1 0	0.84 1 0 1	
student	1 1 0 0	0.91 1 0 1	

상대 위치 인코딩

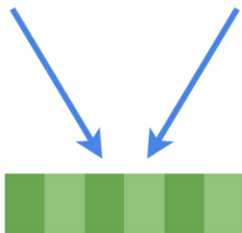
✓ 단어와 단어 사이의 거리를 임베딩

- 위치 편향을 bias로 더함
- 위치 임베딩의 일관성 유지

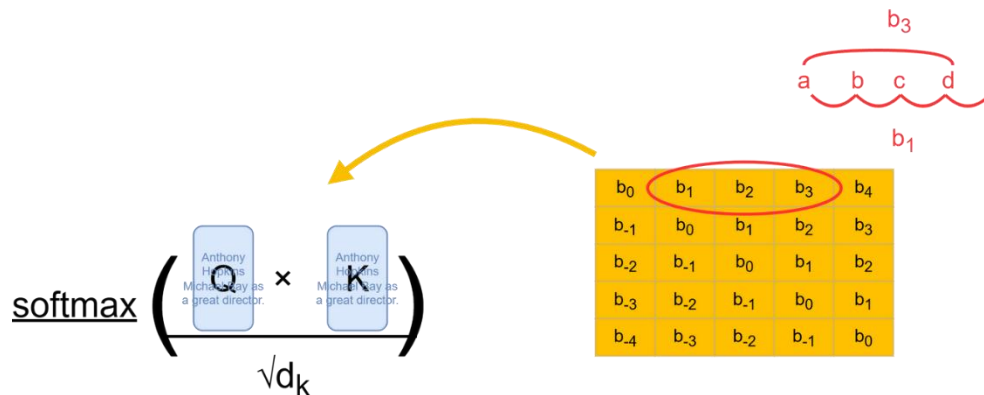
✓ 단점

- 속도가 느림: 위치를 계산해서 더해야 함, 매 스텝마다 값이 변해서 캐싱이 어려움

The **dog** chased the **pig**

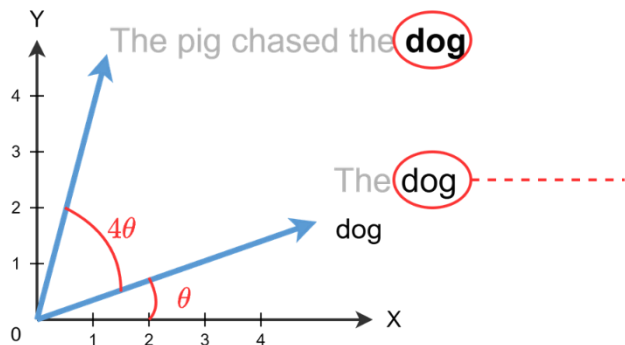


Distance = 3

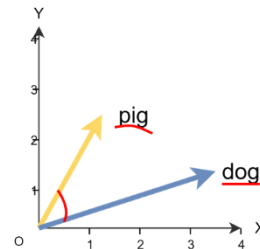


✓ ROPE(Rotary Positional Embeddings)

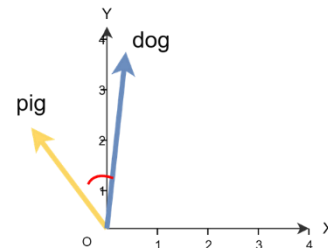
- 위치(m)와 각도(θ)에 비례하여 벡터를 회전(rotation)시키는 방식
- 상대적인 거리가 보존됨
- 효율적인 캐싱(KV cache)가 가능함
- 사인함수 임베딩보다 학습 속도가 빠름



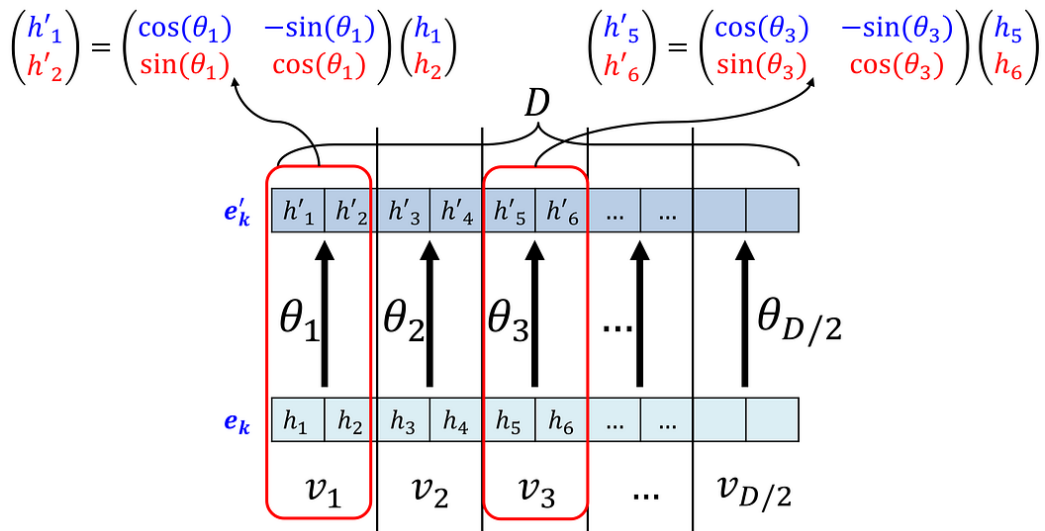
The **pig** chased the **dog**



Once upon a time, the **pig** chased the **dog**

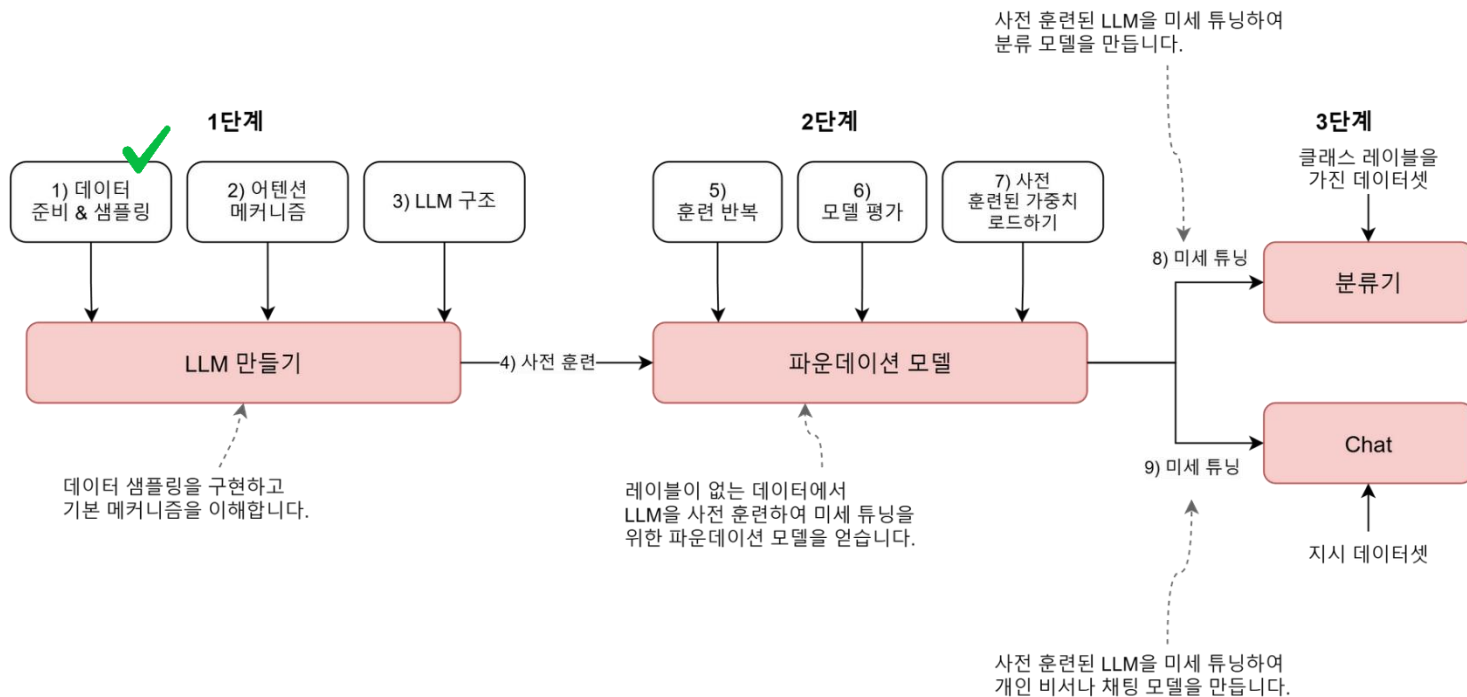


- 2개씩 짝지어서 각도를 다르게 회전함



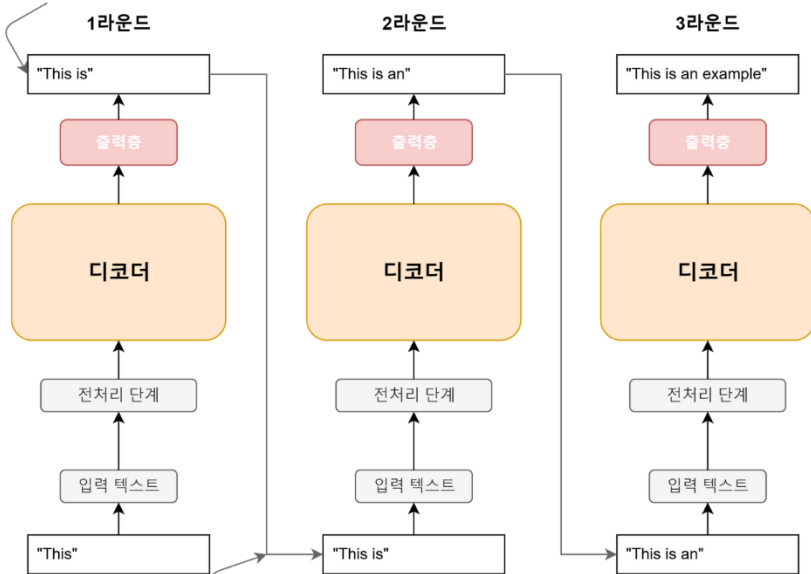
학습

✓ 파운데이션 모델 -> Chat 모델



✓ 다음 단어를 예측: Autoregressive

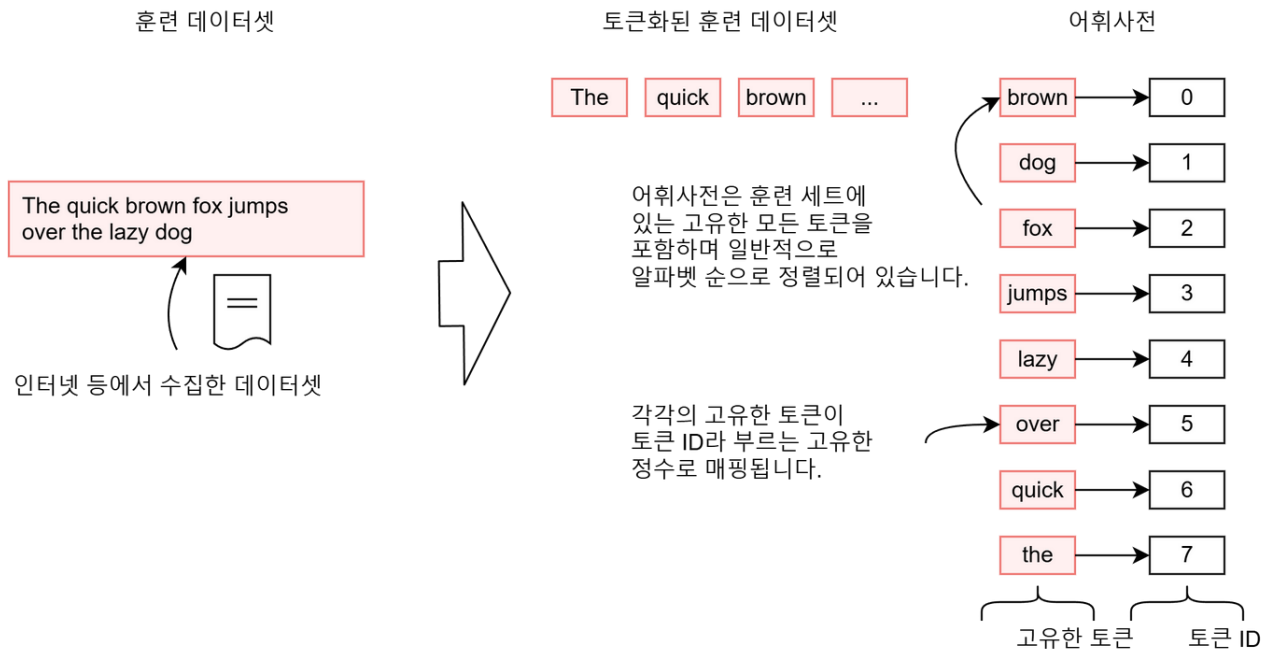
입력 텍스트를 기반으로
다음 단어를 생성합니다.



이전 라운드(round)의 출력이
다음 라운드의 입력으로 사용됩니다.

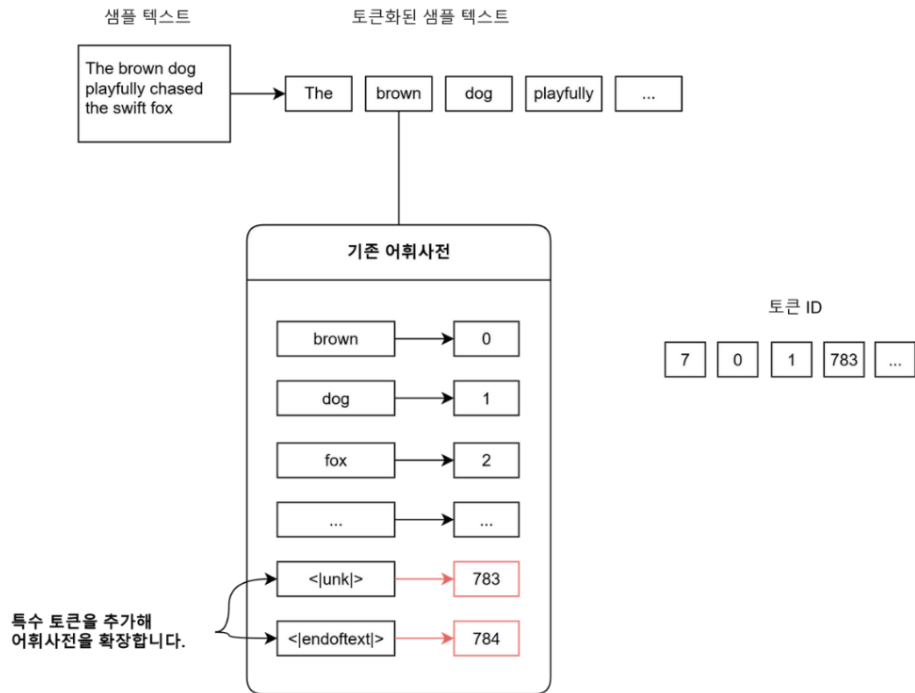
The model is simply trained to
predict the next **word**

✓ 데이터셋을 Token으로 변경



Special Token

- ✓ 모델을 제어하고 입력·출력의 구조를 정의하는 제어 신호
 - 학습 대상에 포함(단 Padding Token은 제외)



Special Token

- ✓ 모델마다 조금씩 다를 수 있음

1. 문장의 시작과 끝 Sequence Control

<BOS>
문장의 시작

<EOS>
문장의 끝 (생성 중지)

<PAD>
길이 맞추기 (패딩)

2. 대화 구조 제어 Chat/Instruction

<|user|>
사용자 질문 시작

<|assistant|>
AI 답변 시작

<|system|>
시스템 프롬프트 (페르소나)

3. 기능 수행 및 사고 Task-Specific

<|thought|>
사고 과정 (CoT)

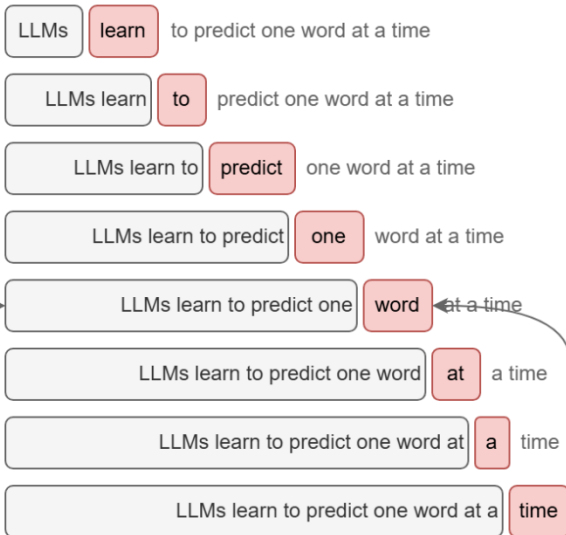
<|file_separator|>
파일 경계 구분

<|tool_call|>
외부 도구 호출

✓ Self-Supervised : Label 없이 생성

LLM은 타깃 이후의 단어를
참조하지 못합니다.

텍스트 샘플:



샘플 텍스트

"In the heart of the city stood the old library, a relic from a bygone era. Its stone walls bore the marks of time, and ivy clung tightly to its facade ..."

입력을
담은 텐서

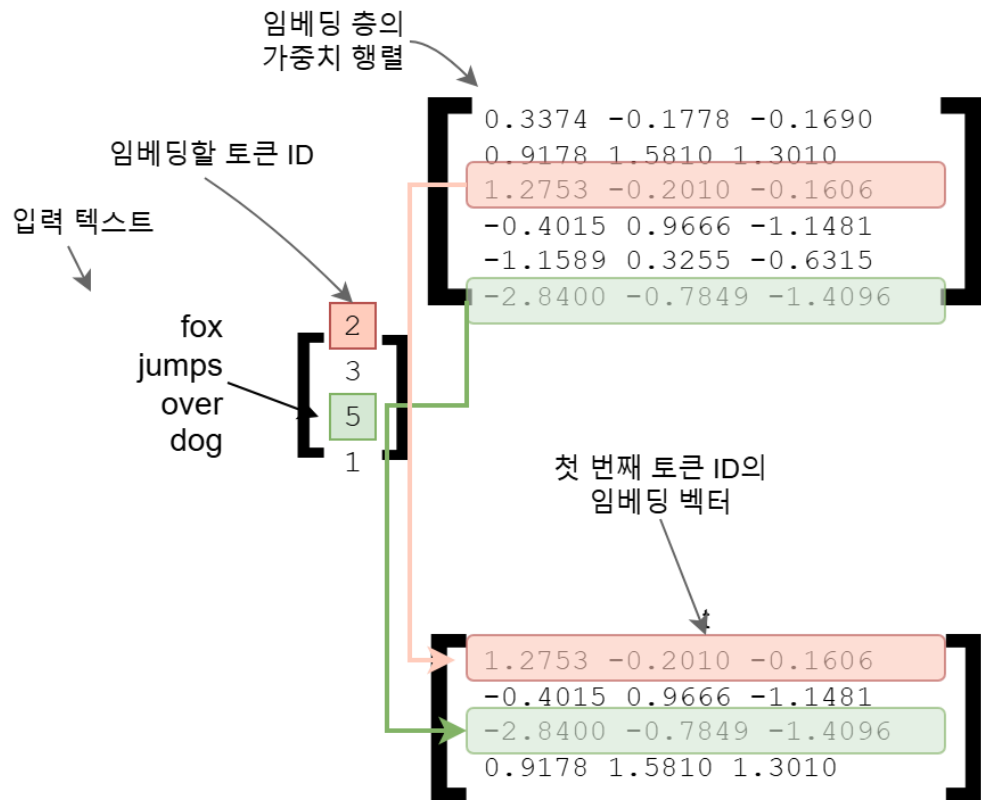
```
x = tensor([["In", "the", "heart", "of"],  
            ["the", "city", "stood", "the"],  
            ["old", "library", "", "a"],  
            [...]])
```

타깃을
담은 텐서

```
y = tensor([["the", "heart", "of", "the"],  
            ["city", "stood", "the", "old"],  
            ["library", "", "a", "relic"],  
            [...]])
```

Token 임베딩

- Token id를 연속 벡터 공간의 점으로 맵핑

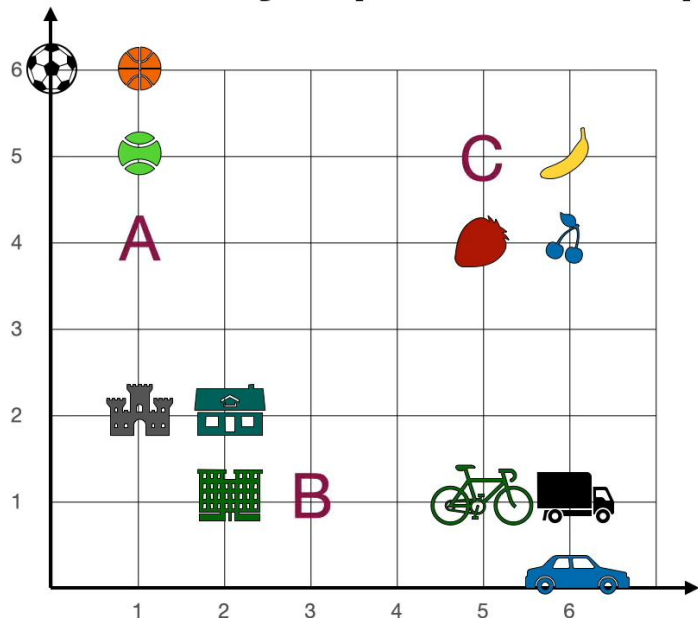




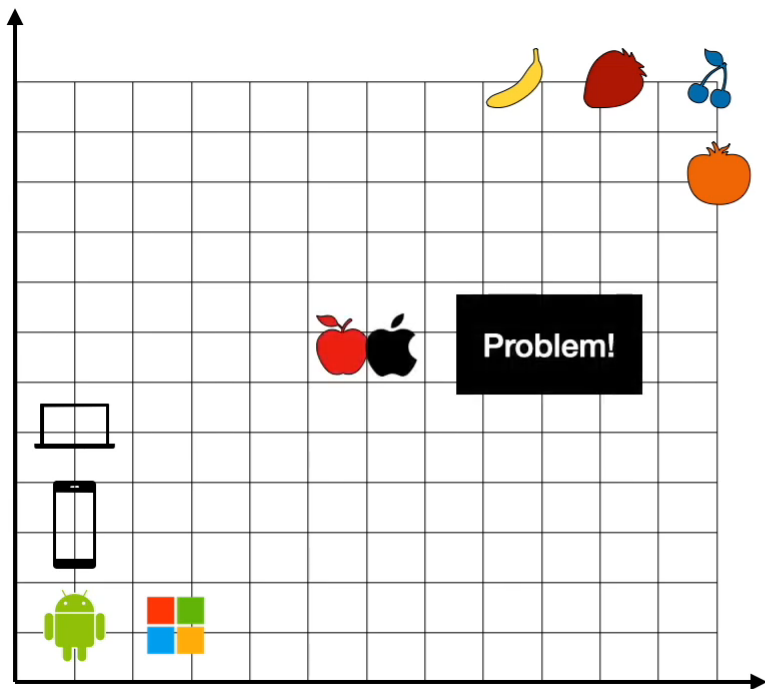
Chapter_2_Exercise_Dataset.ipynb



Where would you put the word apple?



Word	Numbers	
Apple	?	?
Banana	6	5
Strawberry	5	4
Cherry	6	4
Soccer	0	6
Basketball	1	6
Tennis	1	5
Castle	1	2
House	2	2
Building	2	1
Bicycle	5	1
Truck	6	1
Car	6	0



Top right or bottom left?

Cherry 

Android 

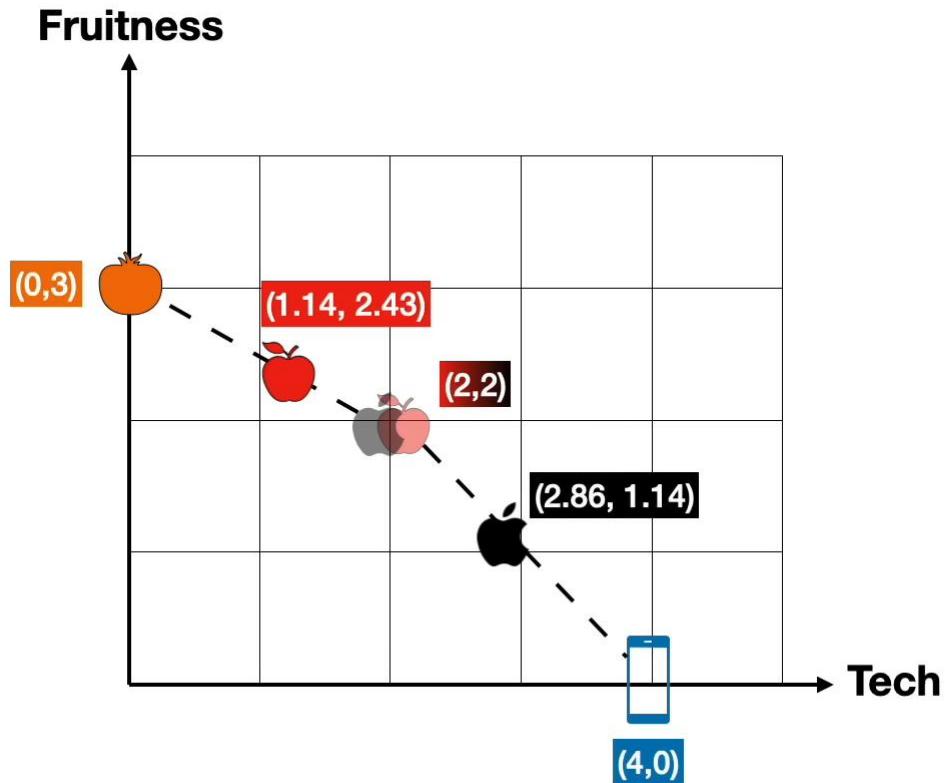
Laptop 

Banana 

Apple?  

GPT: Attention

- ✓ 문맥상에서 어떤 의미를 가질까



an **apple** and an orange

$$\text{Apple} \rightarrow 0.43 \text{ Orange} + 0.57 \text{ Apple}$$

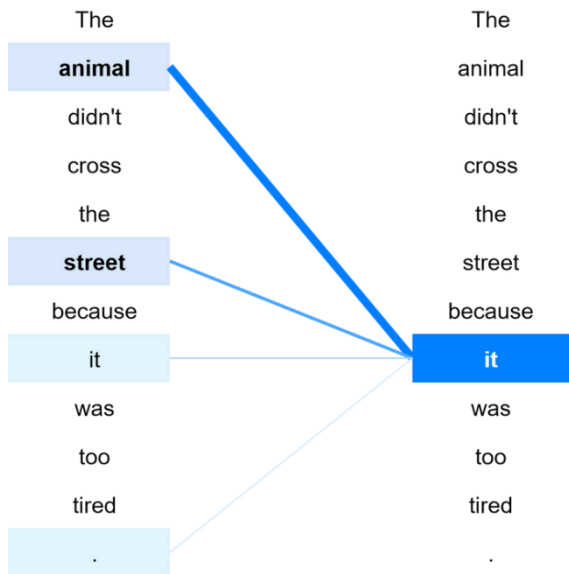
an **apple** phone

$$\text{Apple} \rightarrow 0.43 \text{ Phone} + 0.57 \text{ Apple}$$

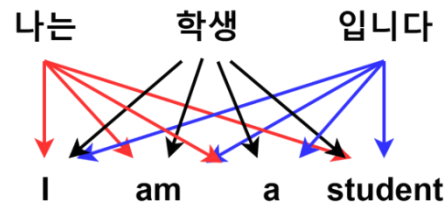
Attention

✓ 문맥을 파악

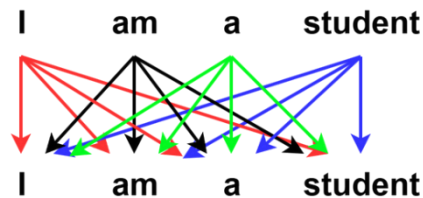
- 연관되어 있으면 값이 크게 나타남



글의 문맥 이해 (Context)



<Attention>

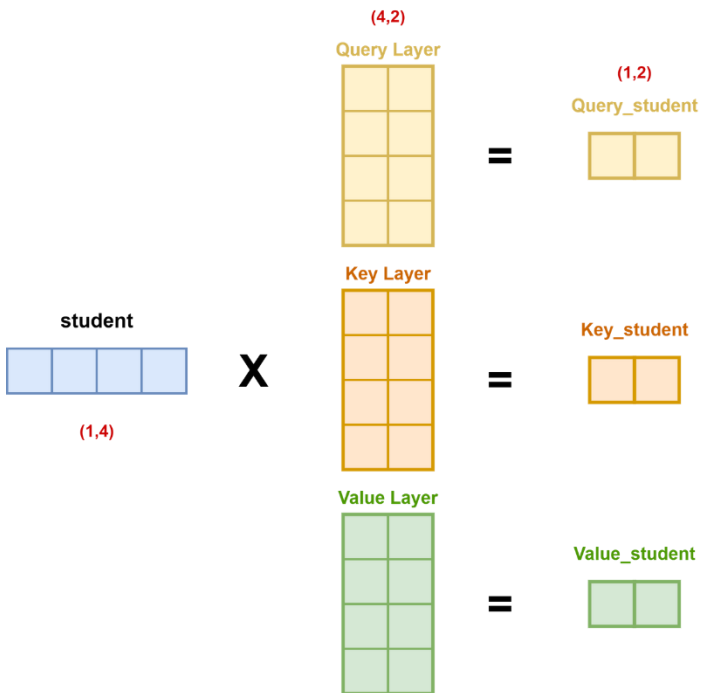


<Self-Attention>



Attention

- ✓ Embedding에서 Weight를 곱해서 Q,K,V를 구함



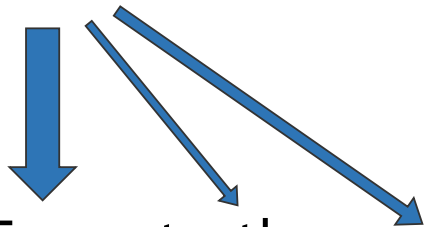
- 질문을 던지는 주체(Query)
- 질문에 답하는 대상(Key)
- 전달할 정보(Value)

Attention

✓ Q,K를 분리하지 않으면 Attention이 제대로 되지 않음

- 항상 자기 자신을 우선 주목함
- 순서를 변경해도 동일한 내적이 나옴

Tom ate the meat

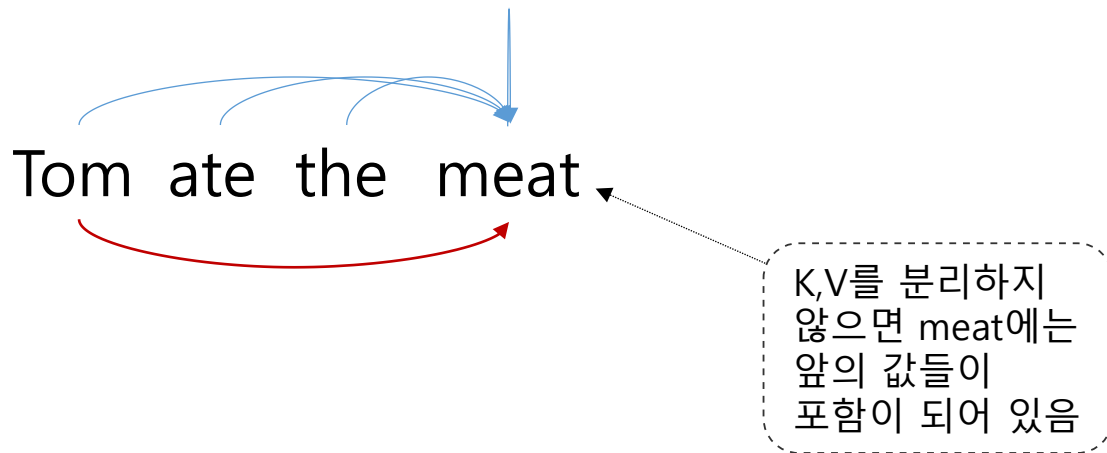


Tom ate the meat

Tom@meat = meat@Tom

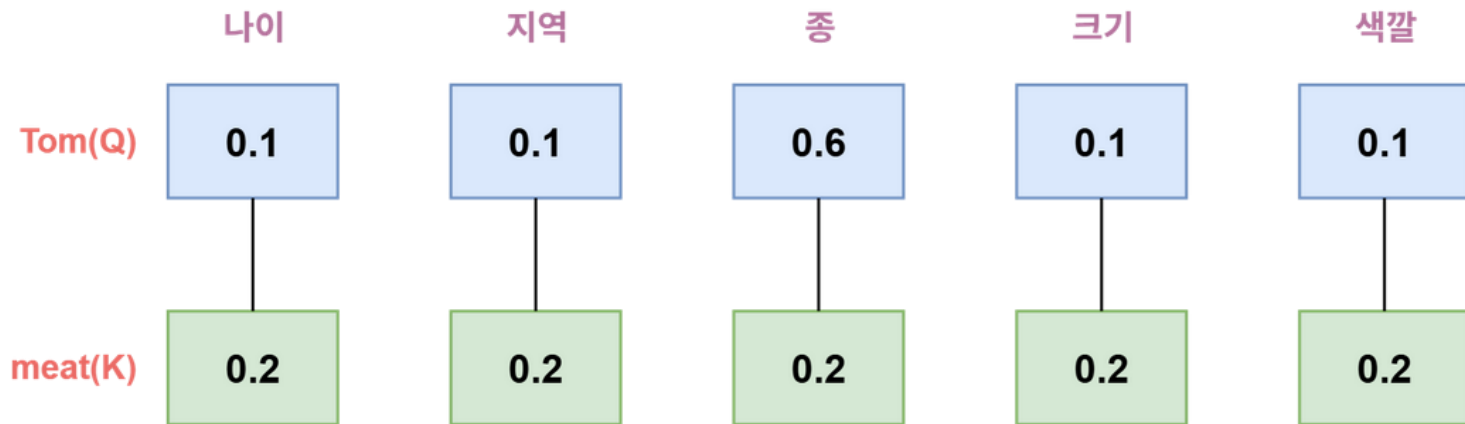
Attention

- ✓ K,V를 분리하지 않으면 전달할 정보에 다른 값이 섞일 수 있음



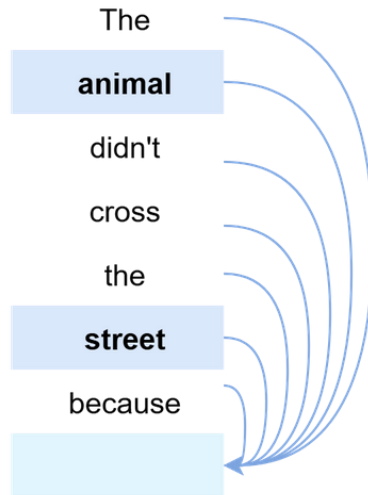
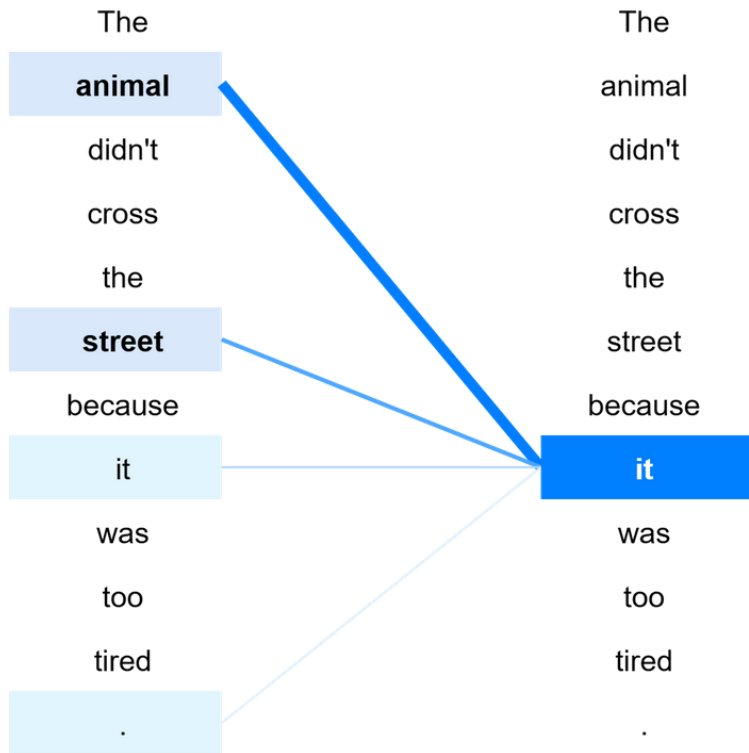
Attention

- ✓ Query는 값을 이용하여 질문을 한다.



Attention

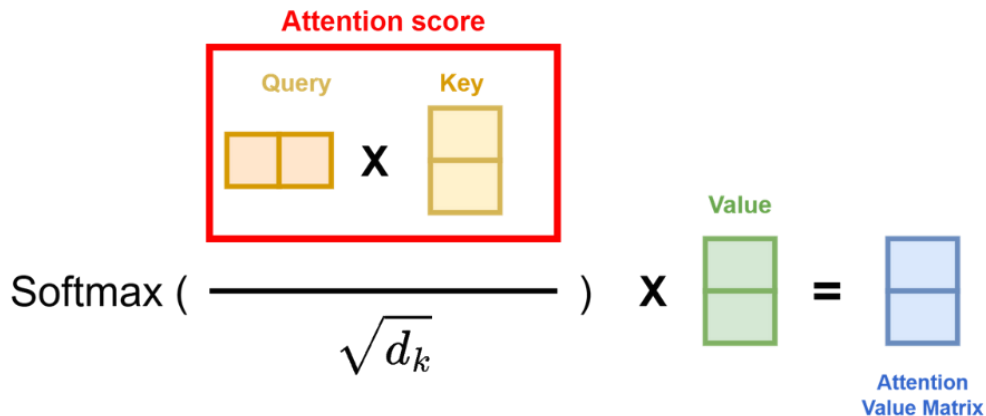
- 다음에 올 단어를 예측하는 것을 학습함으로써 Attention Score가 의미를 가짐



Attention Score

- ✓ 연관되어 있으면 값이 크게 나타남

$$Attention(Q, K, V) = softmax \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$



Attention Map

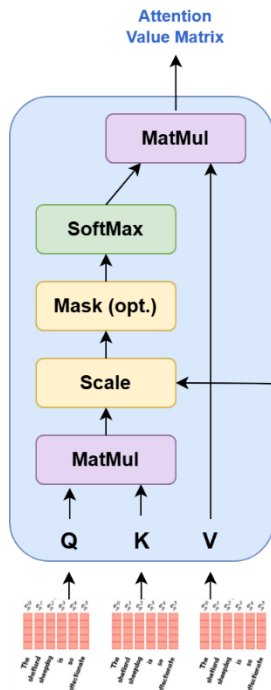
- ✓ Attention Score를 시각화 한 행렬, 어떤 단어와 어떤 단어 연관되어 있는지 시각화

	I	am	a	student
I	1.0	0.5	0.3	1.0
am	0.5	1.0	0.4	0.2
a	0.3	0.4	1.0	0.1
student	1.0	0.2	0.1	1.0

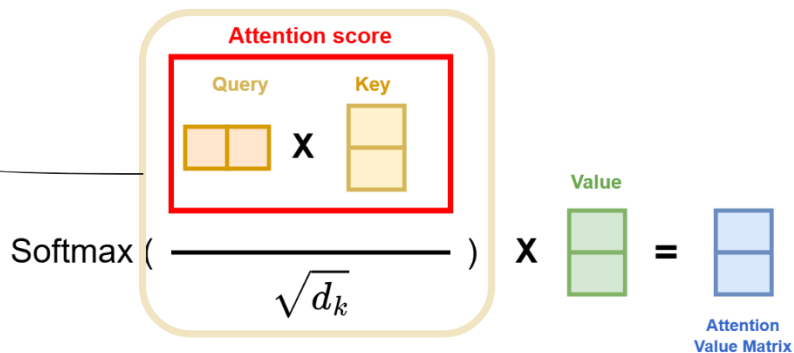
I가 Student와 밀접한 연관이 있다는 것을 보여줌

✓ d_k 로 나누어 분산을 다시 1로 맞춤

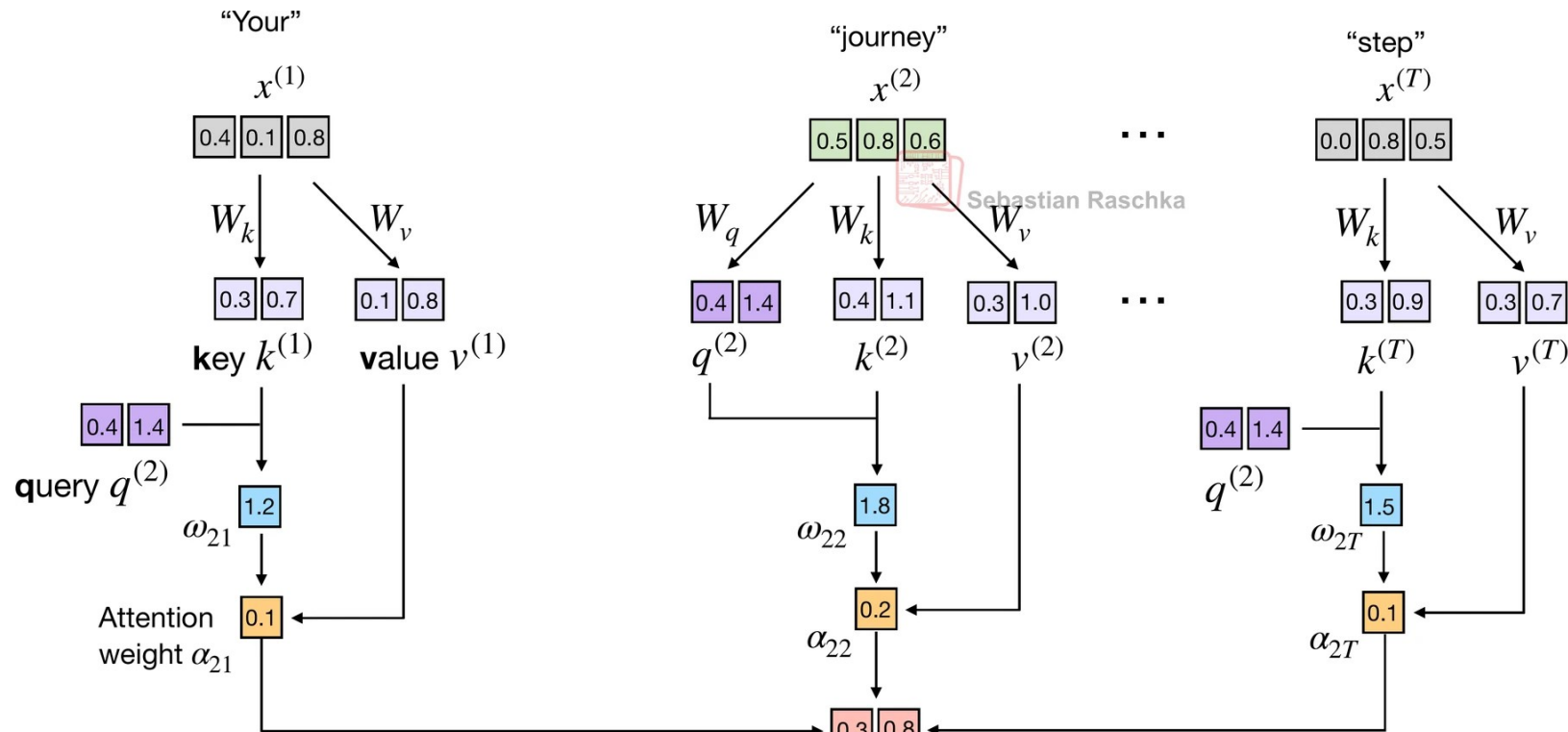
- Attention 연산을 하면 차원 수(d_k)가 커질수록 결과값(Dot-product)이 매우 커지는 경향



$$Attention(Q, K, V) = softmax \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$



전체 흐름



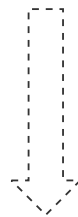
The last step is multiplying each value vector with its respective attention weight and then summing them to obtain the context vector

Attention 의미

- ✓ 문맥의 의미를 더해서 새로운 문맥의 의미를 만들



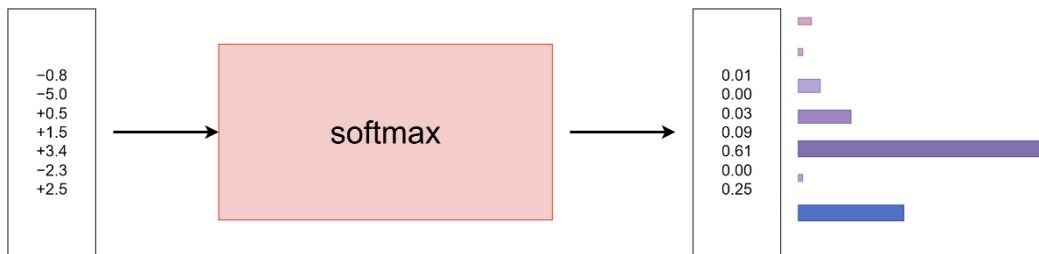
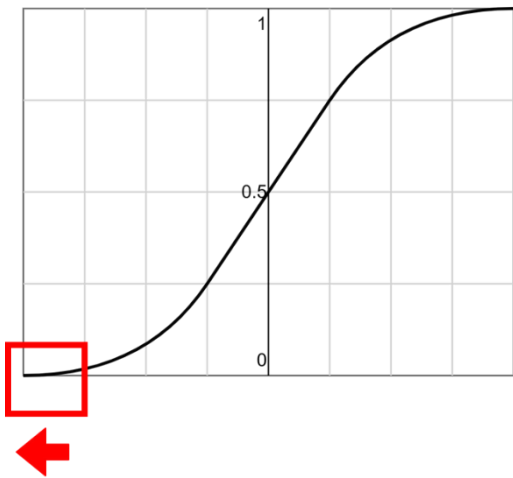
creature



A fluffy blue creature

- 어떤 숫자의 나열을 확률분포로 변환: 지수 함수를 사용

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{i=1}^K e^{z_i}}$$



Masking

- 다음 단어가 학습되지 않도록 가림, $-\infty$ 로 치환 후 softmax

	Your	journey	starts	with	one	step
Your	0.19	0.16	0.16	0.15	0.17	0.15
journey	0.20	0.16	0.16	0.14	0.16	0.14
starts	0.20	0.16	0.16	0.14	0.16	0.14
with	0.18	0.16	0.16	0.15	0.16	0.15
one	0.18	0.16	0.16	0.15	0.16	0.15
step	0.19	0.16	0.16	0.15	0.16	0.15

Attention weight for input tokens
corresponding to “step” and “Your”



	Your	journey	starts	with	one	step
Your	1.0					
journey	0.55	0.44				
starts	0.38	0.30	0.31			
with	0.27	0.24	0.24	0.23		
one	0.21	0.19	0.19	0.18	0.19	
step	0.19	0.16	0.16	0.15	0.16	0.15

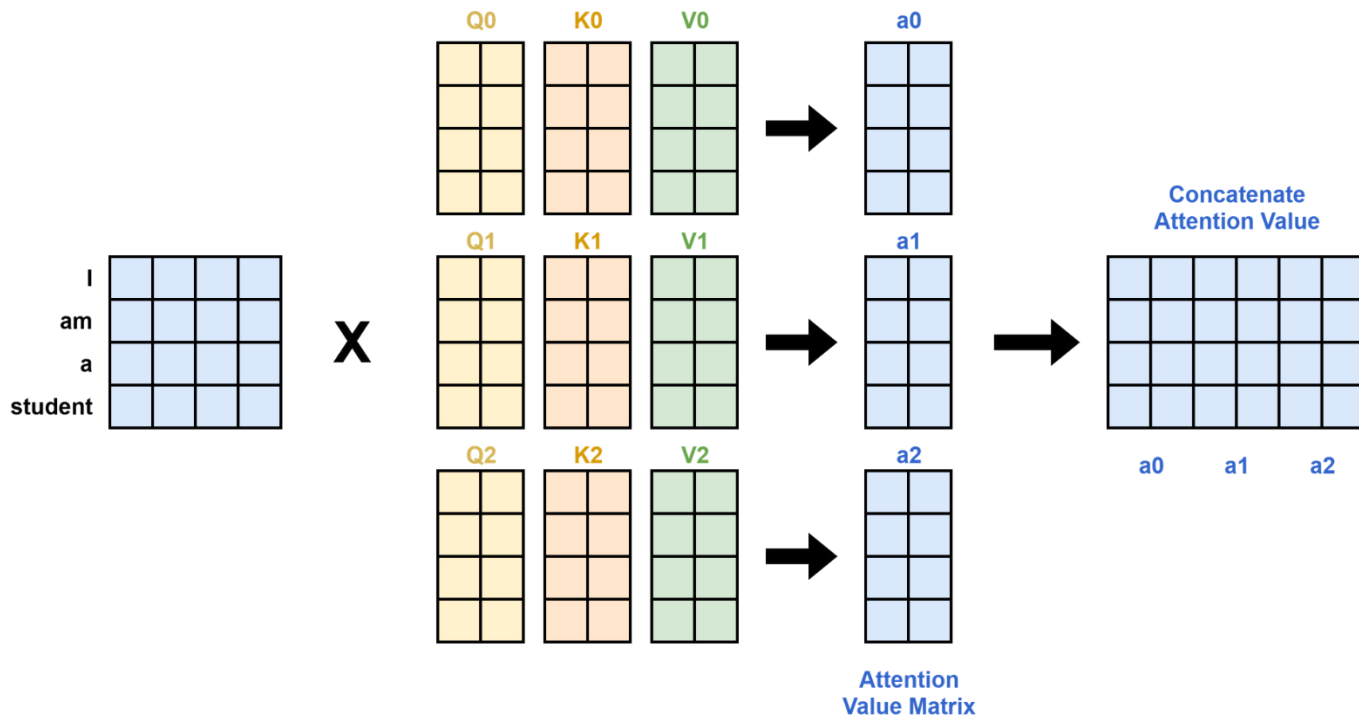
Masked out
future tokens
for the “Your”
token



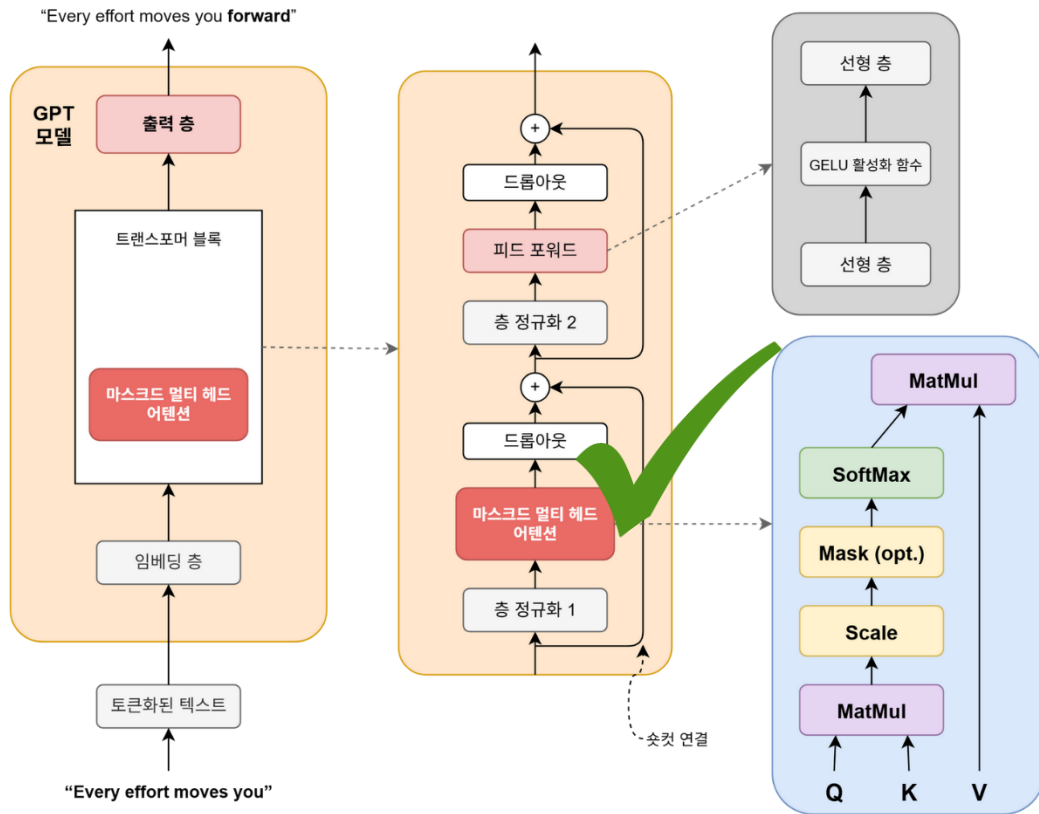
Sebastian Raschka

Multi-head Attention

- ✓ 각각의 Head는 다른 관점에서 문맥을 파악함



✓ Decoder만 사용



Chapter_3_Excercise_Attention.ipynb